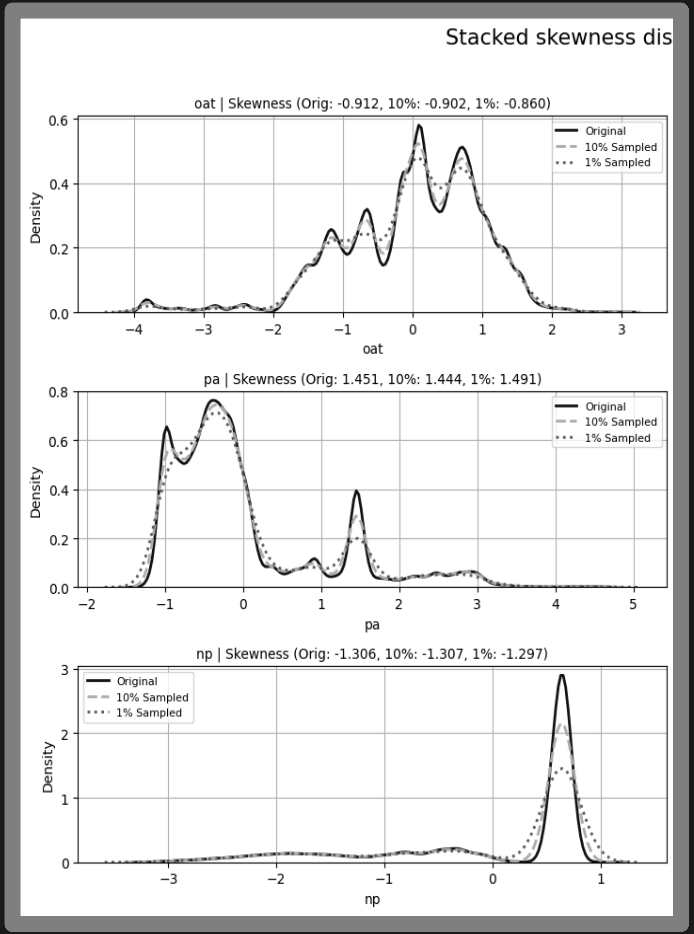
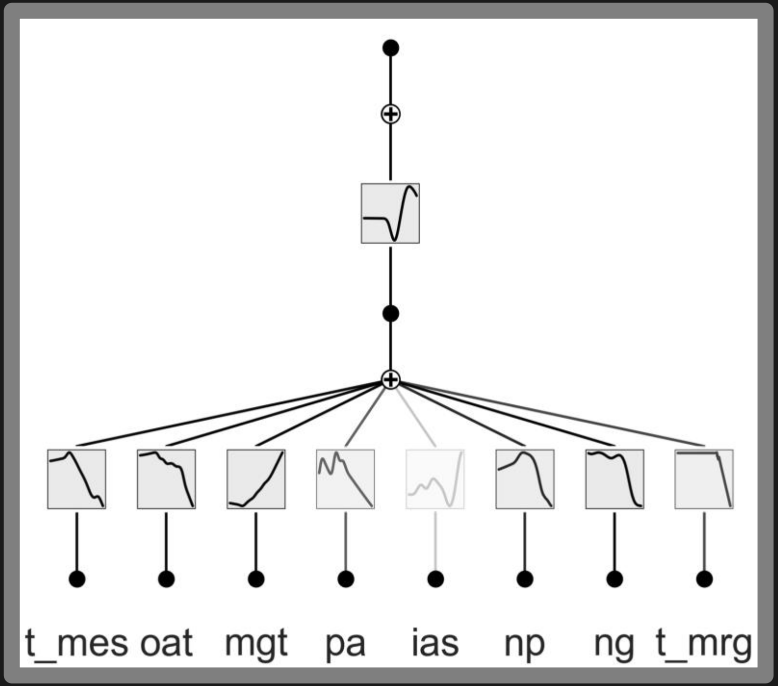

# PHM North America 2024 Challenge (Group A1)

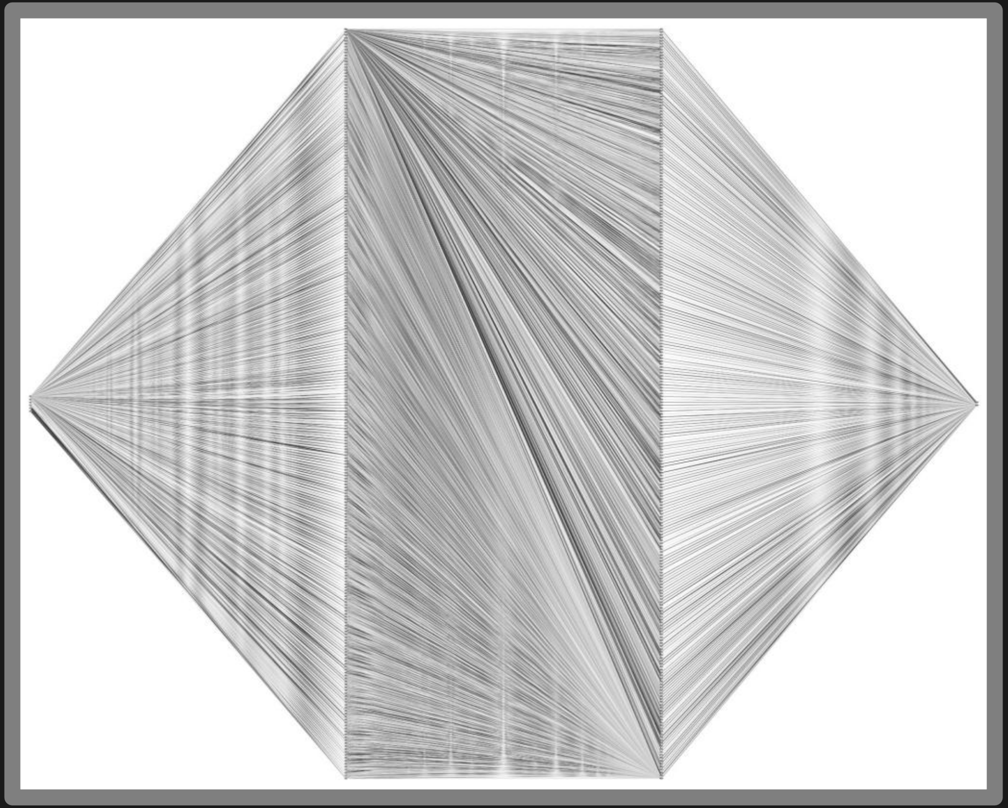
Preventive Maintenance For Robotics And Intelligent Automation Course Examination Project

Valerio Morelli
Federica Paganica
Federico Staffolani
Enrico Maria Sardellini
*a.y. 2024/2025*

# ...Where did we leave off?



$$\phi(x) = w_b b(x) + w_s \sum_{i=1}^{G+k} c_i B_i(c) + \beta$$

# PHM North America 2024 Challenge

**Feature selection**

Which subset of features we chose and how?

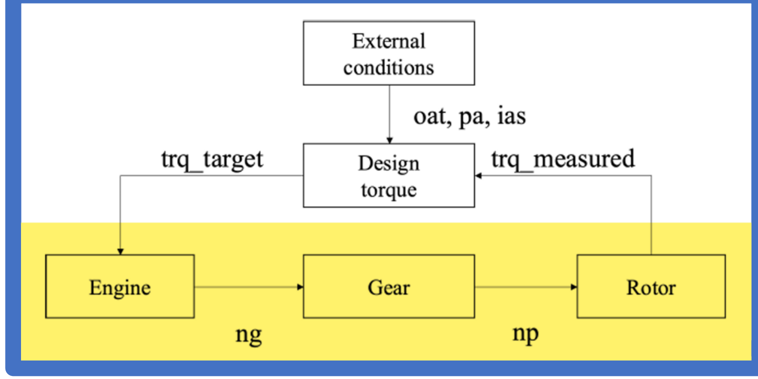
**Probabilistic regression and fault detection**

What are the results of our experiments?

**Qualitative Estimation**

Does the model generalize?

# Multicollinearity checking with *VIF*

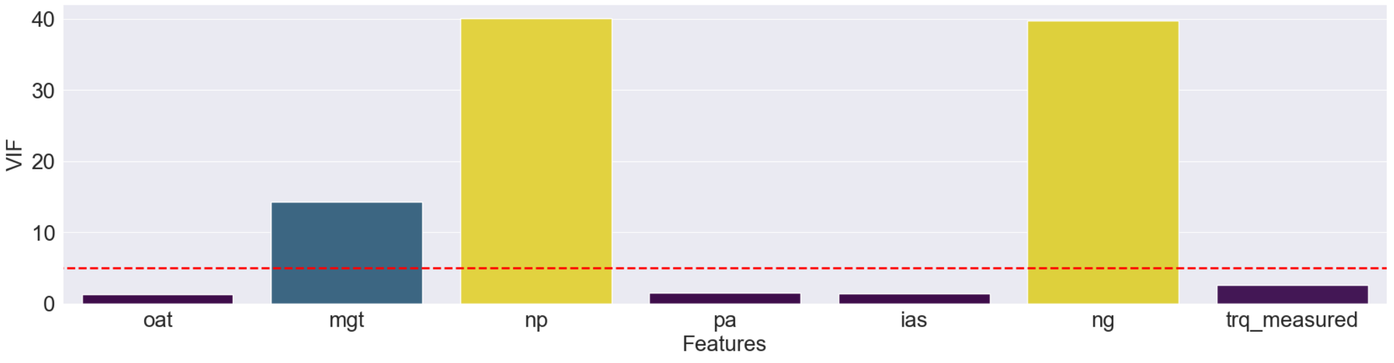


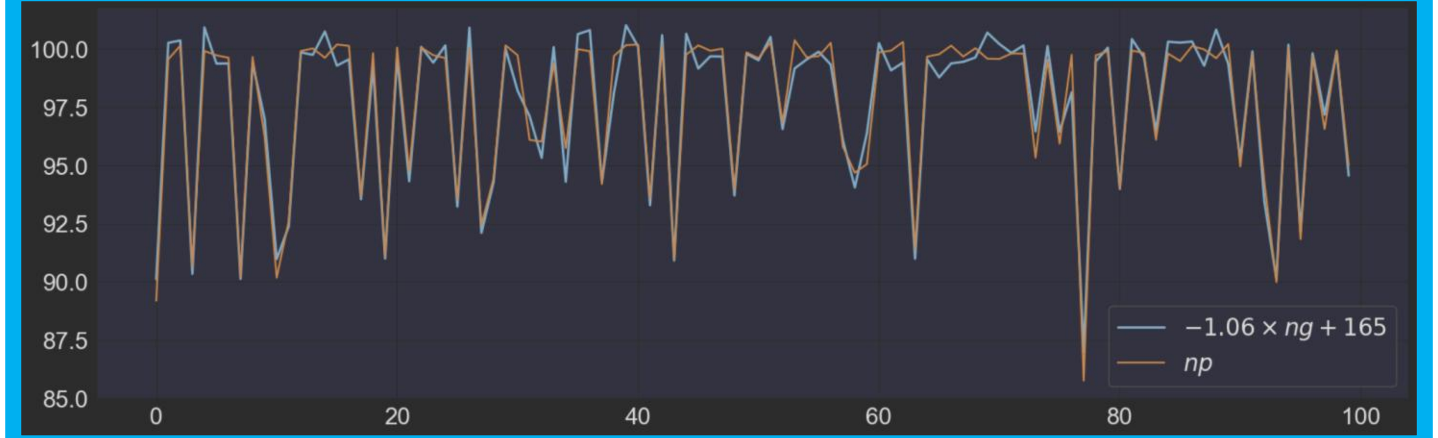

- *np* and *ng* are **strongly inversely correlated** ($\rho = -0.79$)
- *Why inversely?* → **There is a balance of power** between the *compressor-turbine system* and the *power turbine.*

$$\boxed{np \approx -1.06 \times ng + 165}$$

- The *Variance Inflation Factor* measures the collinearity between the features.
- For each of them, **it fits a linear regressor to predict them against the others.**

$$\boxed{VIF(X_i) = \frac{1}{1 - R^2}}$$



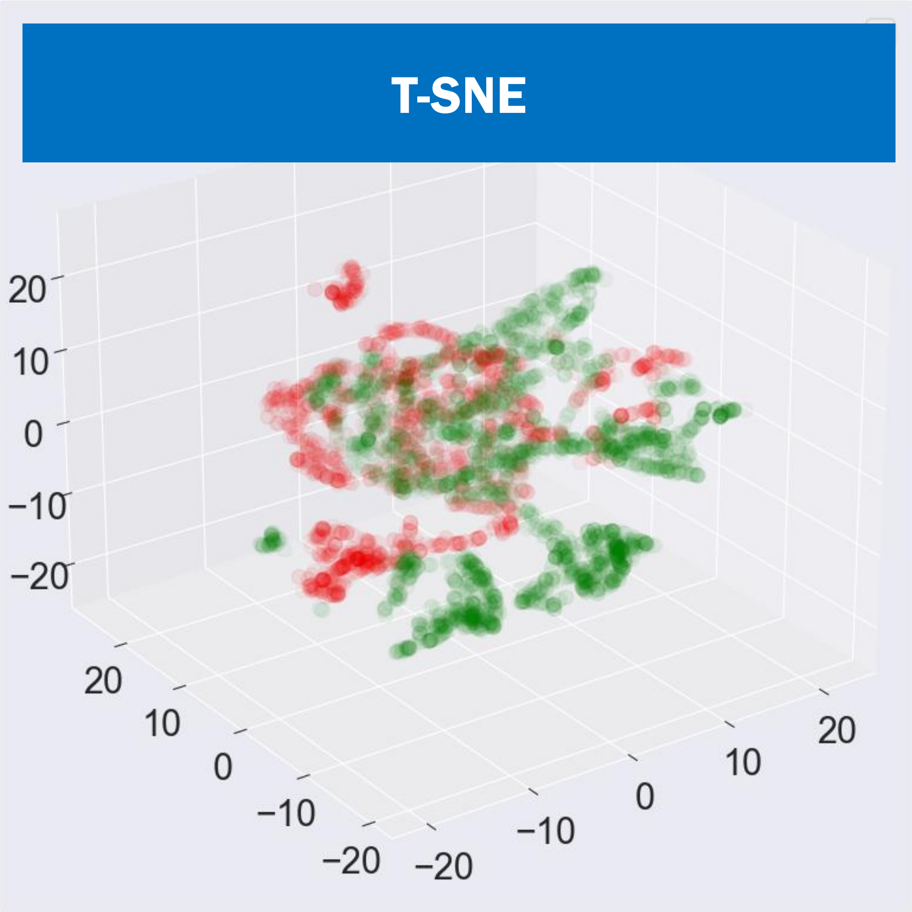
# Feature Selection

- At the end of the feature generation process, **we had 77 features**. This numerosity affect both the **performance** of the model due to the *curse of dimensionality* and the **interpretability** due to a more complex network.

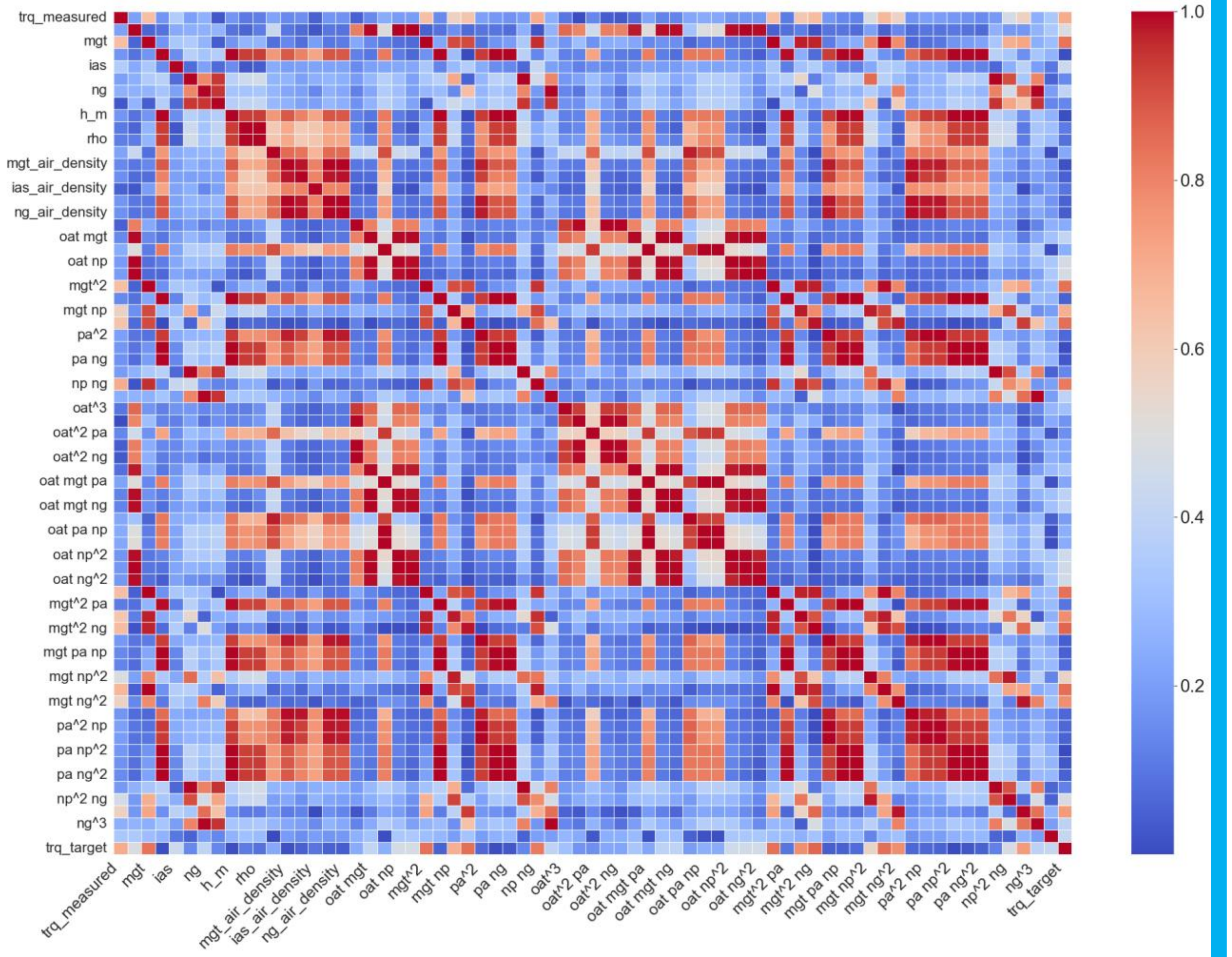
- We evaluated two strategies:

  - **Embedding**: Using *PCA*, or better *t-SNE*, to **replace the feature set with a smaller one**, preserving as much local structures and distribution as possible. However, t-SNE is **too computationally expensive**. Furthermore, the features **lose physics interpretability**

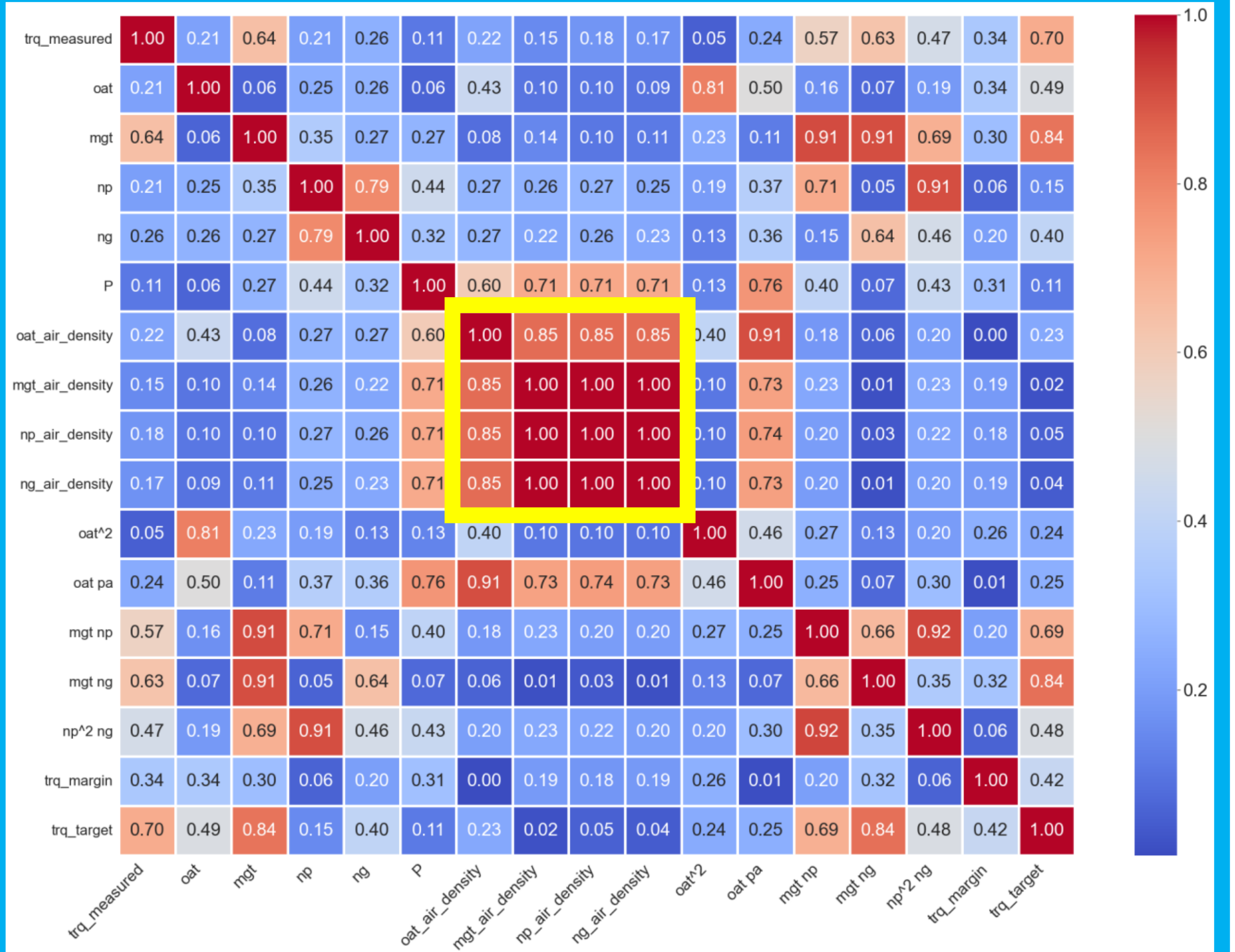
  - **Cherry Picking**: **Combining *Pearson* correlation and *ANOVA*** test to prune less discriminant features. Each captures a different aspect of the features.



# Culling of correlated features




- First of all, **we remove highly correlated features**, because they're likely to cause information ***redundancy***



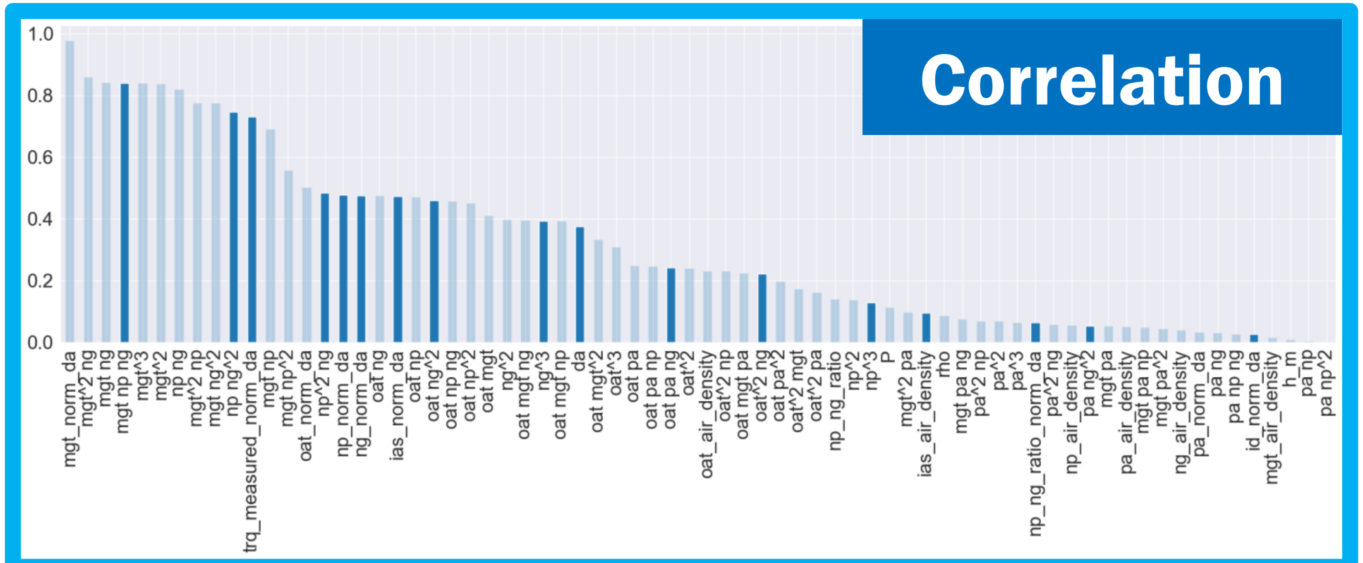
| | trq_measured | oat | mgt | np | ng | P | oat_air_density | mgt_air_density | np_air_density | ng_air_density | oat^2 | oat pa | mgt np | mgt ng | np^2 ng | trq_margin | trq_target |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| trq_measured | 1.00 | 0.21 | 0.64 | 0.21 | 0.26 | 0.11 | 0.22 | 0.15 | 0.18 | 0.17 | 0.05 | 0.24 | 0.57 | 0.63 | 0.47 | 0.34 | 0.70 |
| oat | 0.21 | 1.00 | 0.06 | 0.25 | 0.26 | 0.06 | 0.43 | 0.10 | 0.10 | 0.09 | 0.81 | 0.50 | 0.16 | 0.07 | 0.19 | 0.34 | 0.49 |
| mgt | 0.64 | 0.06 | 1.00 | 0.35 | 0.27 | 0.27 | 0.08 | 0.14 | 0.10 | 0.11 | 0.23 | 0.11 | 0.91 | 0.91 | 0.69 | 0.30 | 0.84 |
| np | 0.21 | 0.25 | 0.35 | 1.00 | 0.79 | 0.44 | 0.27 | 0.26 | 0.27 | 0.25 | 0.19 | 0.37 | 0.71 | 0.05 | 0.91 | 0.06 | 0.15 |
| ng | 0.26 | 0.26 | 0.27 | 0.79 | 1.00 | 0.32 | 0.27 | 0.22 | 0.26 | 0.23 | 0.13 | 0.36 | 0.15 | 0.64 | 0.46 | 0.20 | 0.40 |
| P | 0.11 | 0.06 | 0.27 | 0.44 | 0.32 | 1.00 | 0.60 | 0.71 | 0.71 | 0.71 | 0.13 | 0.76 | 0.40 | 0.07 | 0.43 | 0.31 | 0.11 |
| oat_air_density | 0.22 | 0.43 | 0.08 | 0.27 | 0.27 | 0.60 | 1.00 | 0.85 | 0.85 | 0.85 | 0.40 | 0.91 | 0.18 | 0.06 | 0.20 | 0.00 | 0.23 |
| mgt_air_density | 0.15 | 0.10 | 0.14 | 0.26 | 0.22 | 0.71 | 0.85 | 1.00 | 1.00 | 1.00 | 0.10 | 0.73 | 0.23 | 0.01 | 0.23 | 0.19 | 0.02 |
| np_air_density | 0.18 | 0.10 | 0.10 | 0.27 | 0.26 | 0.71 | 0.85 | 1.00 | 1.00 | 1.00 | 0.10 | 0.74 | 0.20 | 0.03 | 0.22 | 0.18 | 0.05 |
| ng_air_density | 0.17 | 0.09 | 0.11 | 0.25 | 0.23 | 0.71 | 0.85 | 1.00 | 1.00 | 1.00 | 0.10 | 0.73 | 0.20 | 0.01 | 0.20 | 0.19 | 0.04 |
| oat^2 | 0.05 | 0.81 | 0.23 | 0.19 | 0.13 | 0.13 | 0.40 | 0.10 | 0.10 | 0.10 | 1.00 | 0.46 | 0.27 | 0.13 | 0.20 | 0.26 | 0.24 |
| oat pa | 0.24 | 0.50 | 0.11 | 0.37 | 0.36 | 0.76 | 0.91 | 0.73 | 0.74 | 0.73 | 0.46 | 1.00 | 0.25 | 0.07 | 0.30 | 0.01 | 0.25 |
| mgt np | 0.57 | 0.16 | 0.91 | 0.71 | 0.15 | 0.40 | 0.18 | 0.23 | 0.20 | 0.20 | 0.27 | 0.25 | 1.00 | 0.66 | 0.92 | 0.20 | 0.69 |
| mgt ng | 0.63 | 0.07 | 0.91 | 0.05 | 0.64 | 0.07 | 0.06 | 0.01 | 0.03 | 0.01 | 0.13 | 0.07 | 0.66 | 1.00 | 0.35 | 0.32 | 0.84 |
| np^2 ng | 0.47 | 0.19 | 0.69 | 0.91 | 0.46 | 0.43 | 0.20 | 0.23 | 0.22 | 0.20 | 0.20 | 0.30 | 0.92 | 0.35 | 1.00 | 0.06 | 0.48 |
| trq_margin | 0.34 | 0.34 | 0.30 | 0.06 | 0.20 | 0.31 | 0.00 | 0.19 | 0.18 | 0.19 | 0.26 | 0.01 | 0.20 | 0.32 | 0.06 | 1.00 | 0.42 |
| trq_target | 0.70 | 0.49 | 0.84 | 0.15 | 0.40 | 0.11 | 0.23 | 0.02 | 0.05 | 0.04 | 0.24 | 0.25 | 0.69 | 0.84 | 0.48 | 0.42 | 1.00 |

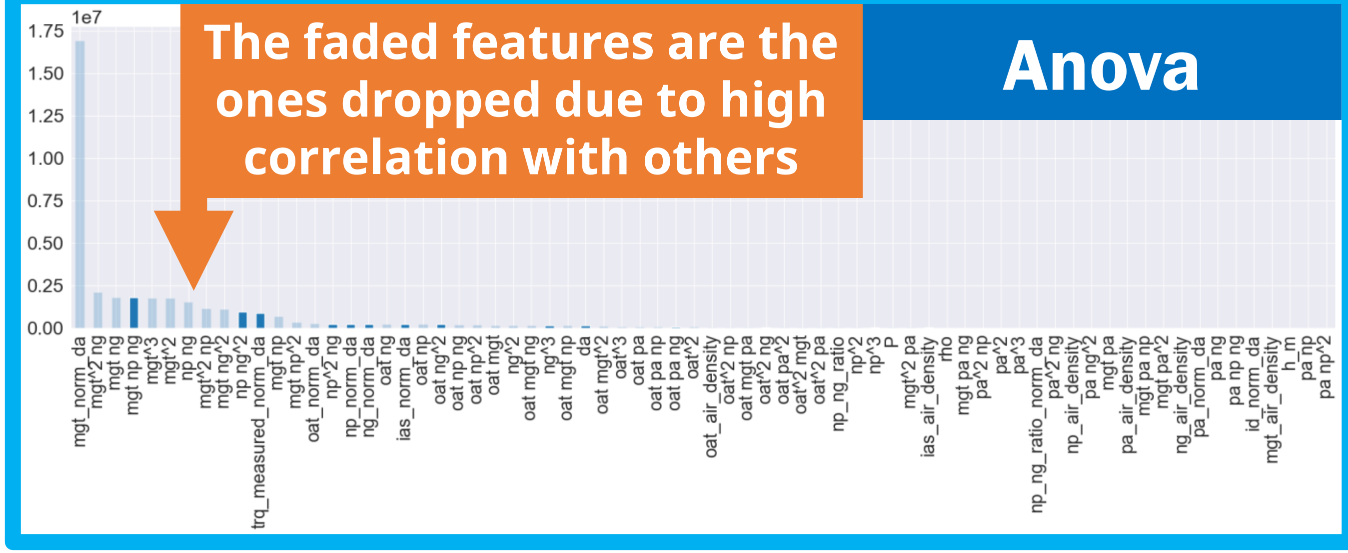


- We used a **threshold of 0.925**.

mgt
np
700
650
600
550
500
0
20
40
60
80
100
$mgt_{ad}$
$np_{ad}$
7000
6000
5000
4000
3000
2000
1000
0
The high correlation is due to small variance of $mgt$ and $np$ compared to that of $ad$

# Correlation with target and ANOVA



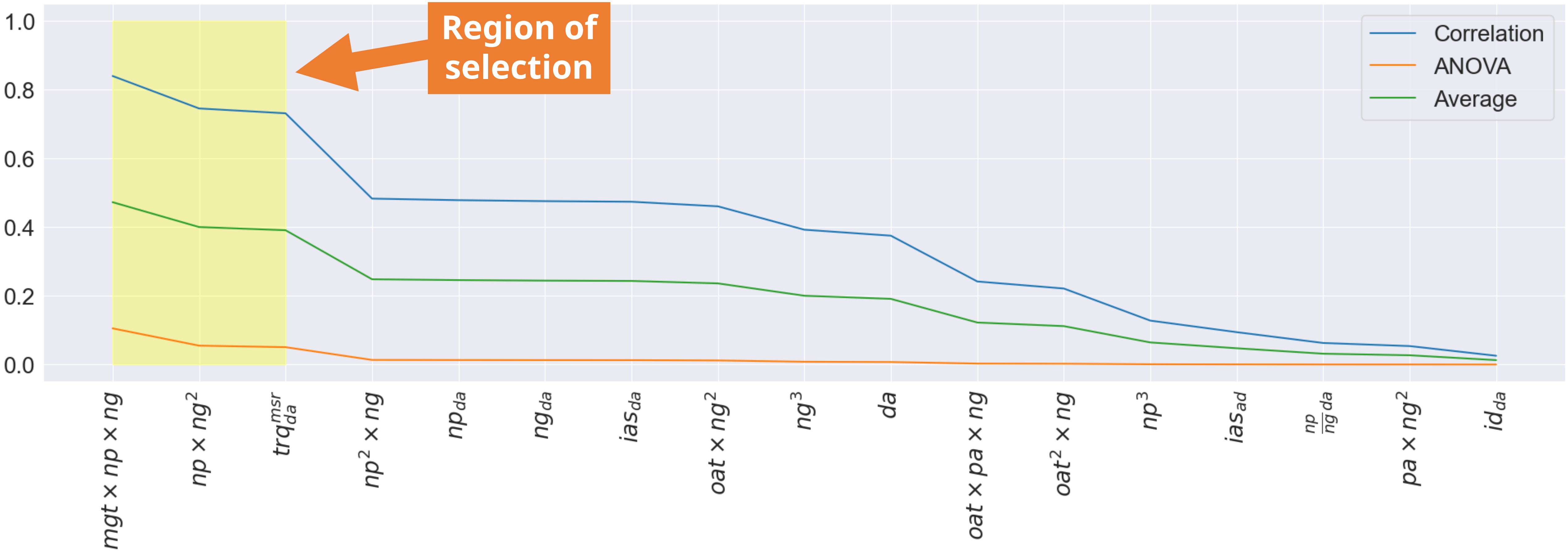




- `f_regression` is the implementation of **ANOVA test for regression tasks**.

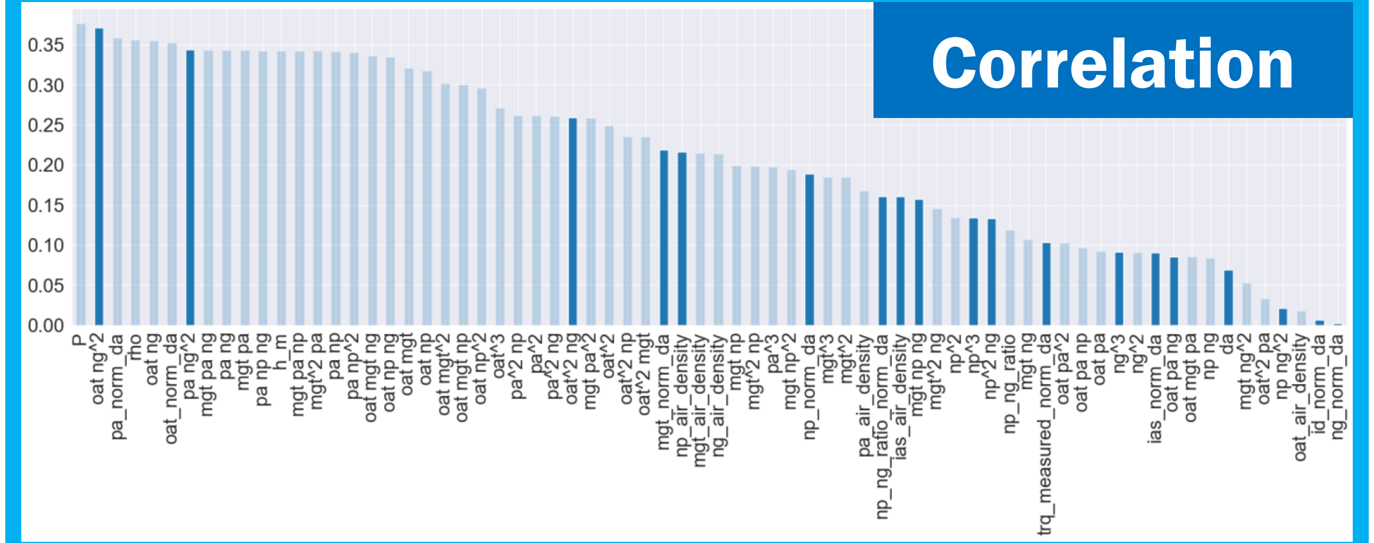
$H_0^i$: The i-th feature has **no linear relationship** with the target.

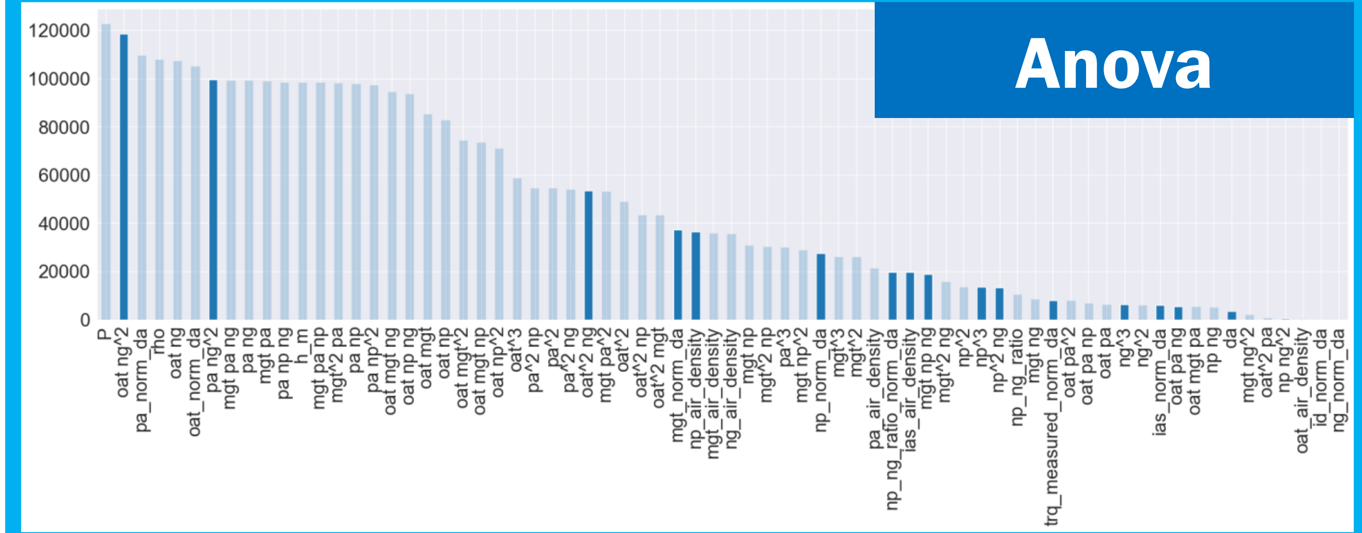
- If the $H_0$ holds, **the cross-correlation** between each feature and the target **follows the *Fisher* distribution**.

# Cherry picking



# Corr. with target and ANOVA



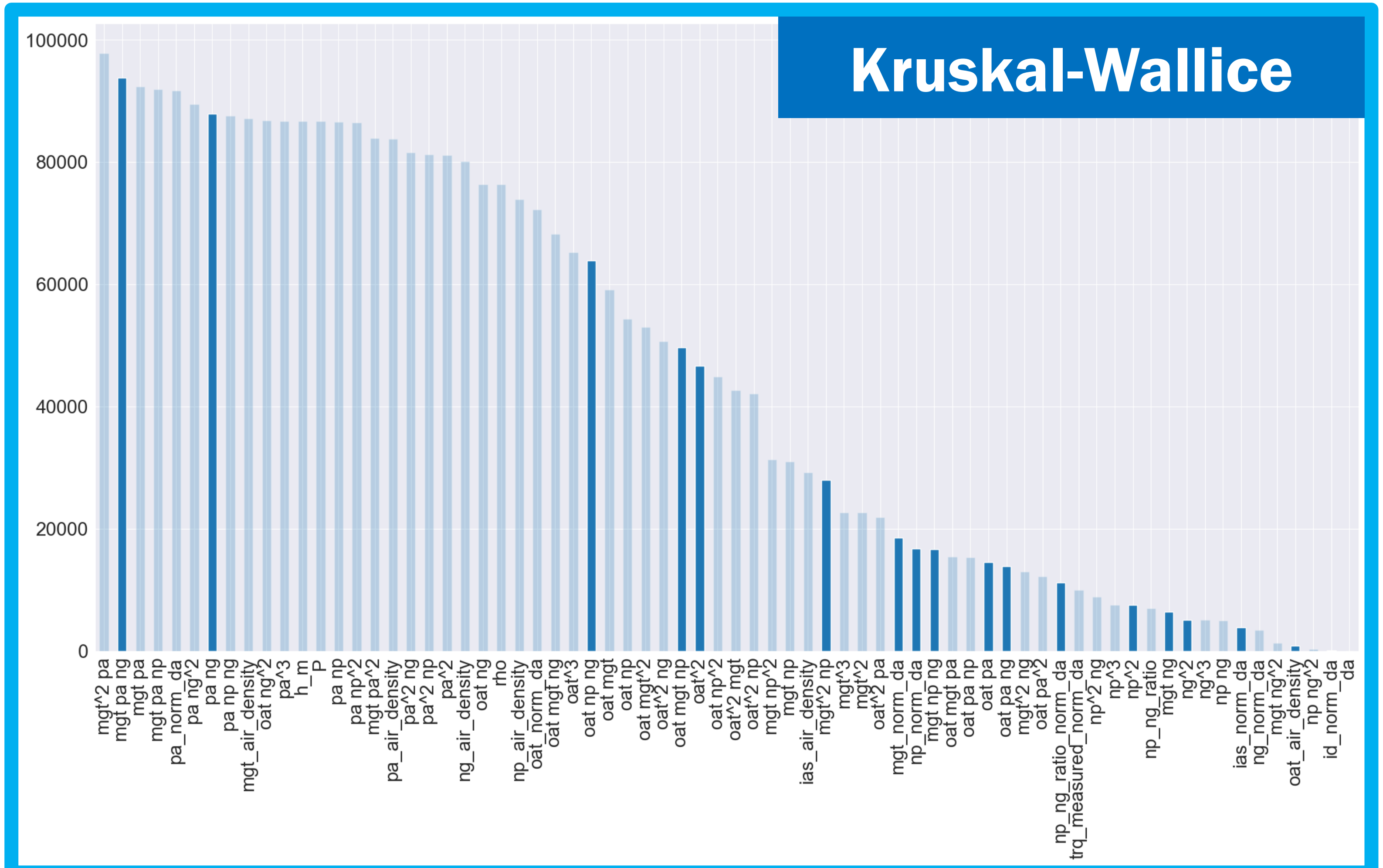

- `f_classif` is the implementation of **ANOVA test for classification tasks**.

$\boldsymbol{H_0^i}$: The **mean of the classes** is equal to that of the i-th feature.

- If $H_0$ holds, the $\frac{MSR}{MSE_i}$ **ratio follows the *Fisher* distribution**.

**Anyway… ANOVA assumes normal distribution**

# Kruskal-Wallice test




$\boldsymbol{H_0^i}$: The **median of the classes** is equal to that of the i-th feature.

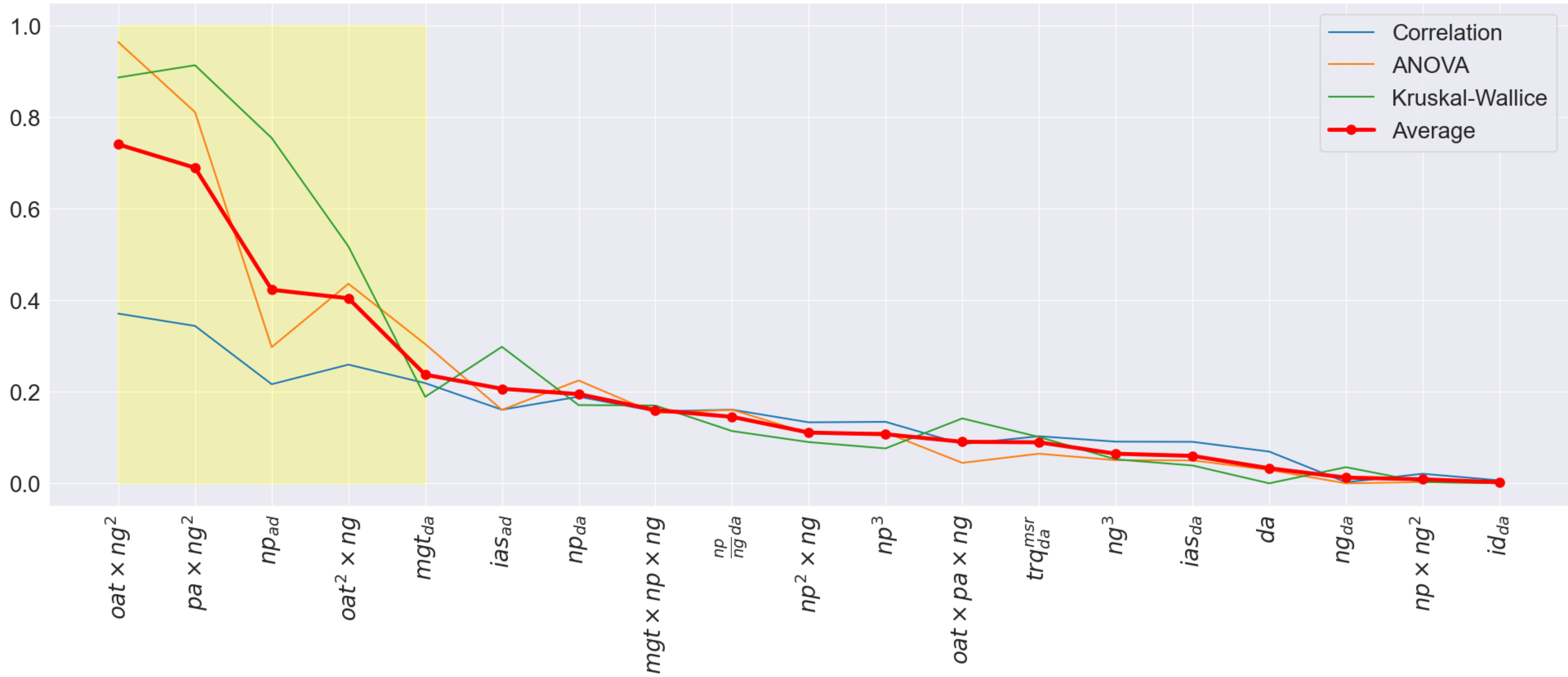
- If $H_0$ holds, the $\frac{MSR}{MSE_i}$ **follows the** $\chi^2$ **distribution**.

---

- We have reason to believe that the classes **doesn't follow the normal** distribution **is skewed and has many outliers** according to the *IQR* rule.
  *~ More in section 3*

**Kruskal-Wallice is a non-parametric variant of ANOVA**

# Cherry picking



# PHM North America 2024 Challenge

### Feature selection

Which subset of features we chose and how?

### Probabilistic regression and fault detection

What are the results of our experiments?

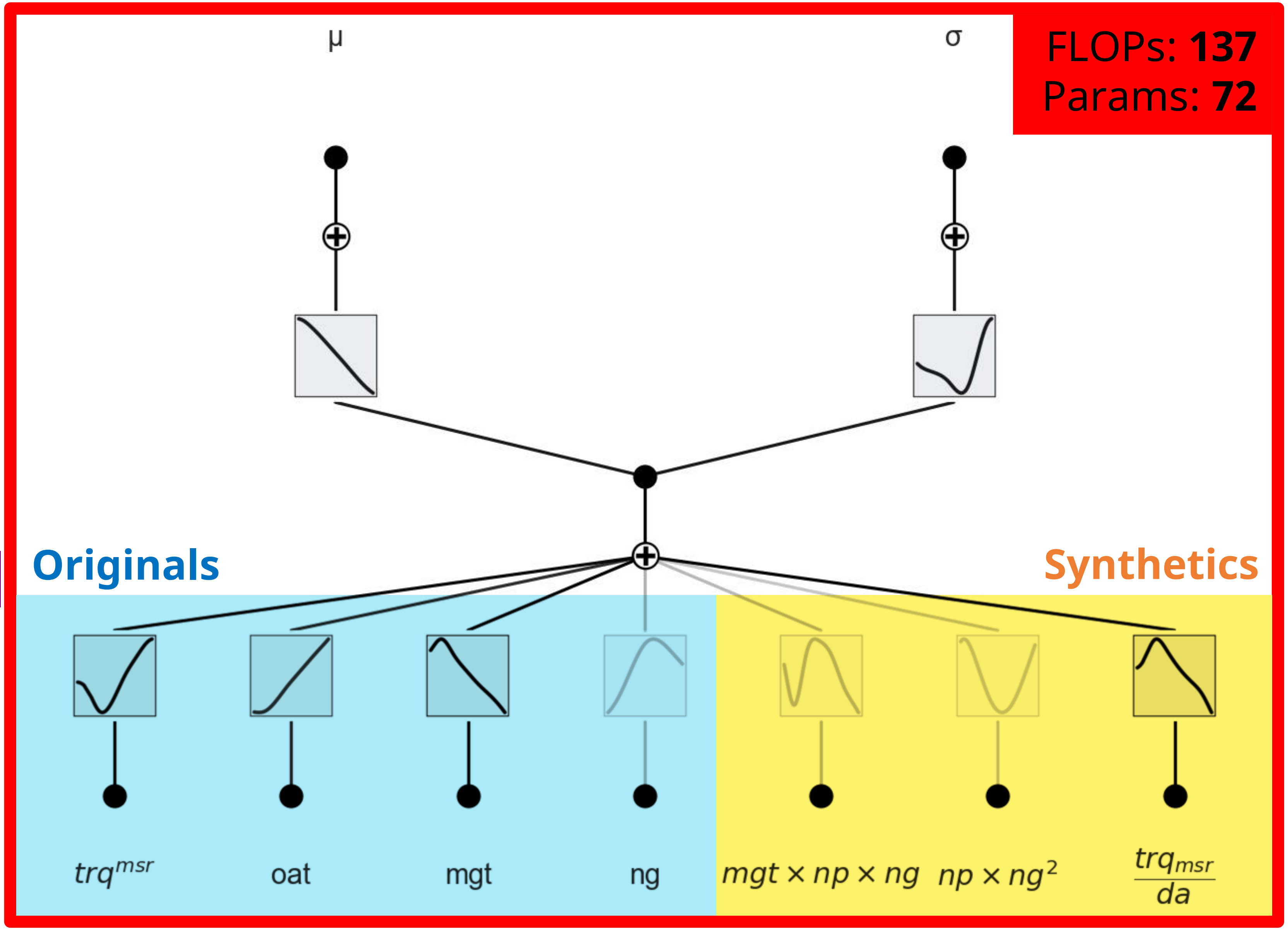
### Qualitative Estimation

Does the model generalize?

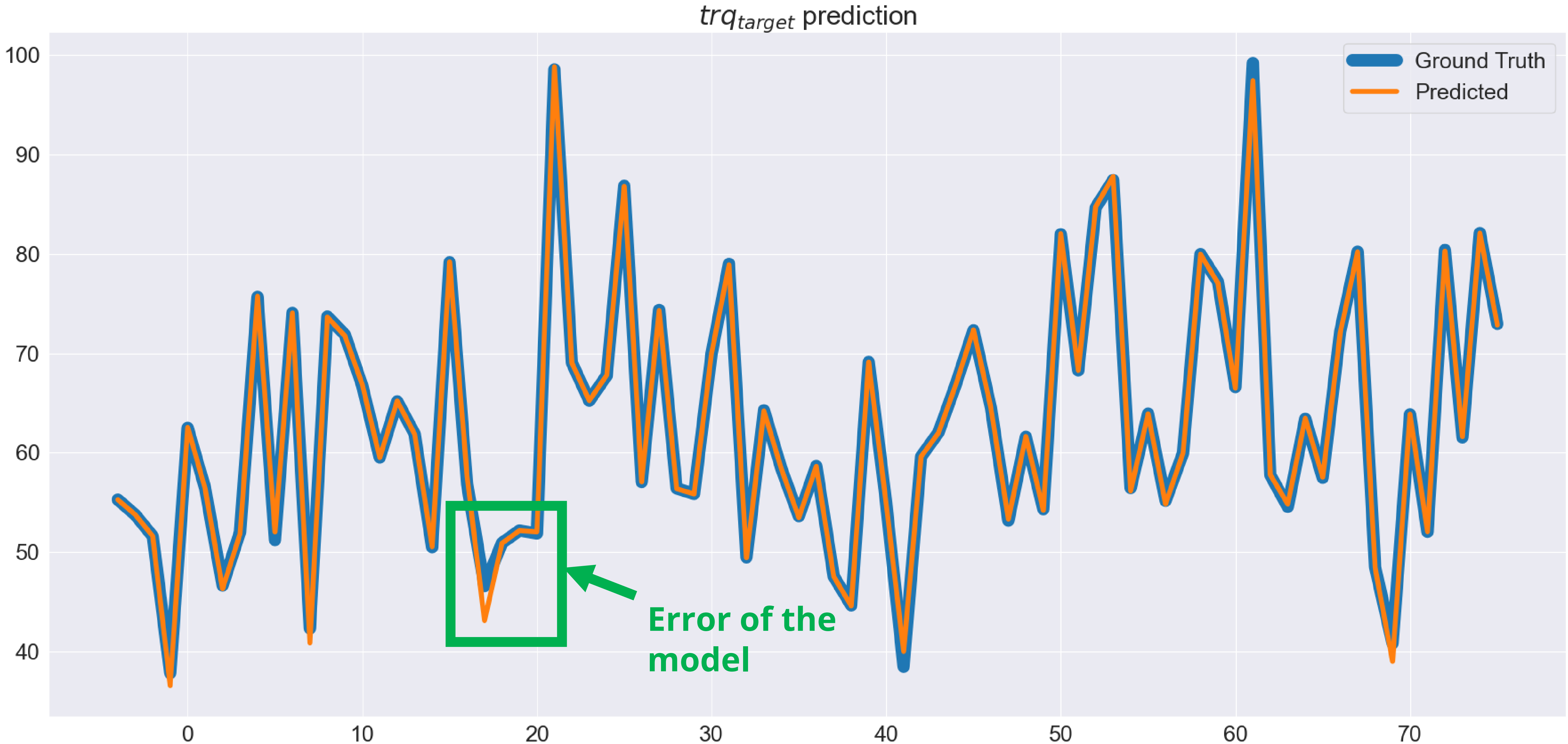
# Probabilistic Regression with PyKAN

- Optimizer:
  $Adam, lr = 0.05$
- Scheduler:
  Exponential, $\gamma = 0.96$
- Epochs: 25
- Batch size: 8192
- Loss: *GaussianNLLLoss*

- **The *Standardization* happens online**, since each training uses a different train set.
- The $\mu$ and $\sigma$ **are stored** to **later *de-standardize* the output** $\boldsymbol{trq_{trq}}$



# Torque target prediction




**Error of the model**

*regression*

# Probabilistic Regression with PyKAN

## Ground Truth

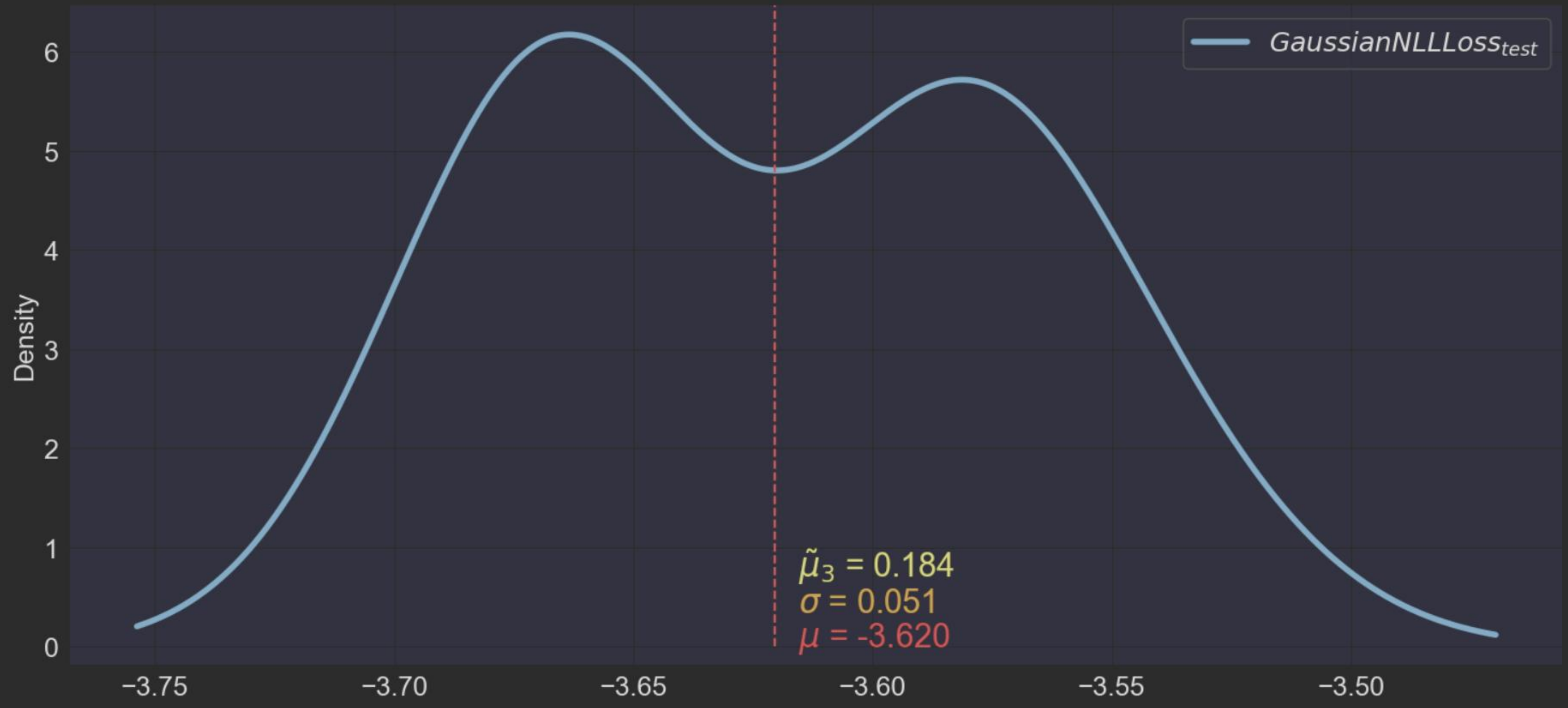
| trq_margin | trq_target |
|---|---|
| -13.71774514924153 | 62.70118936225788 |
| 1.7918630774837023 | 48.751440930229926 |
| -13.94487072380339 | 60.42638066710049 |
| -0.017280748085576 | 62.41078505054281 |
| 7.32240400768298 | 58.60845233721728 |
| -1.00490918707624l | 54.54618967120598 |
| -6.17447736781564 9 | 87.85422951827468 |
| 8.181552971370602 | 55.0925720356156 |
| -17.39993244117741 3 | 81.8585650088587 |
| -15.63974772979118 7 | 97.08363571231564 |
| -4.52516770185869 6 | 53.94091695199825 |

## Predicted

| trq_margin | trq_target |
|---|---|
| -14.018585020926377 | 62.92057418823242 |
| 1.460935277794517 3 | 48.91044998168945 |
| -13.71573787335233 | 60.26591491699219 |
| 0.361421197213696 03 | 62.17528533935547 |
| 7.43833682146592 | 58.54521179199219 |
| -0.93381642480296 9 | 54.50704574584961 |
| -7.92454326118633 5 | 89.52406311035156 |
| 8.07822218532872 | 55.14524459838867 |
| -17.317600927427456 | 81.77705383300781 |
| -15.144869854982323 | 96.51744079589844 |
| -4.744858144769237 | 54.06532287597656 |

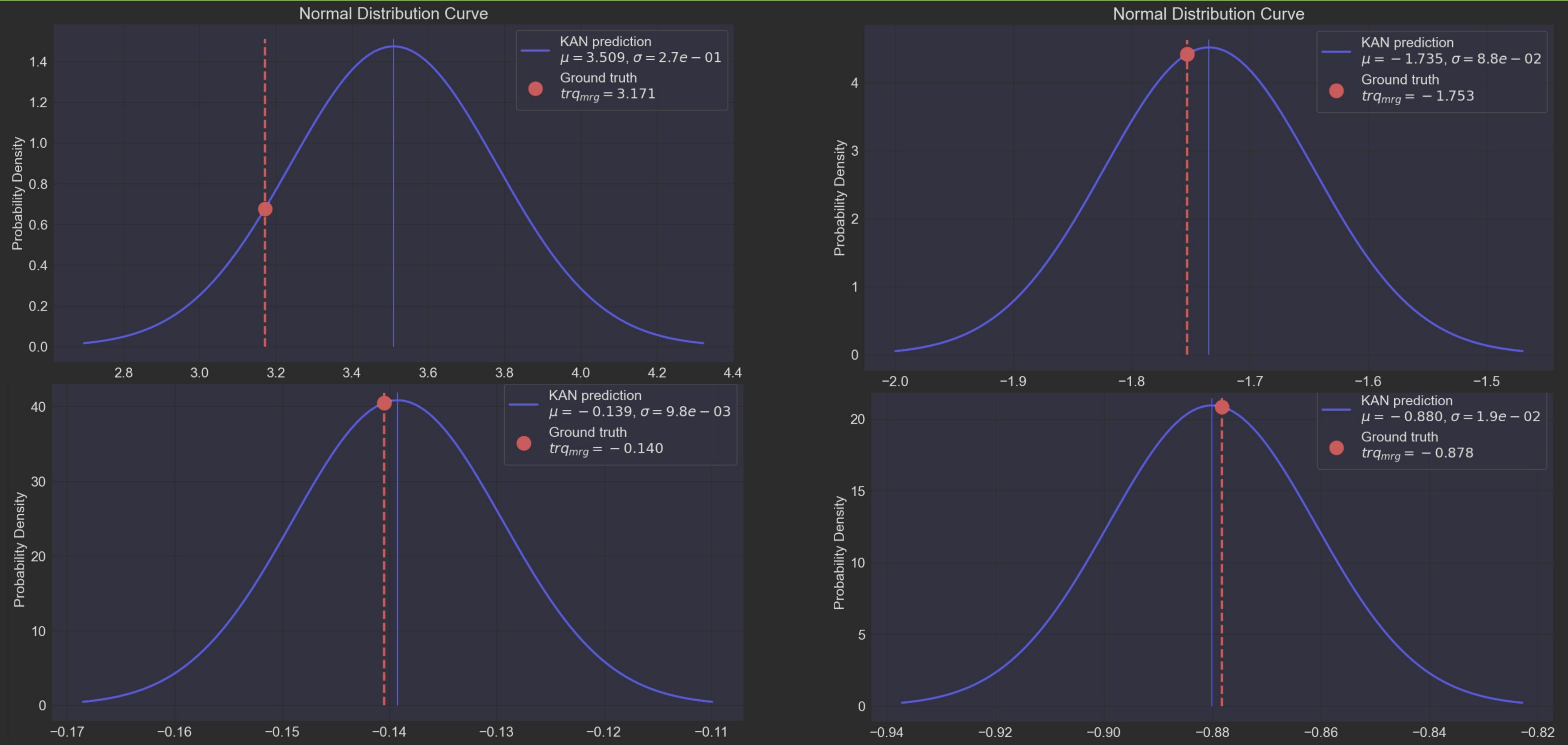
# Results of 10 trainings

## Does model performance depend on the dataset split?

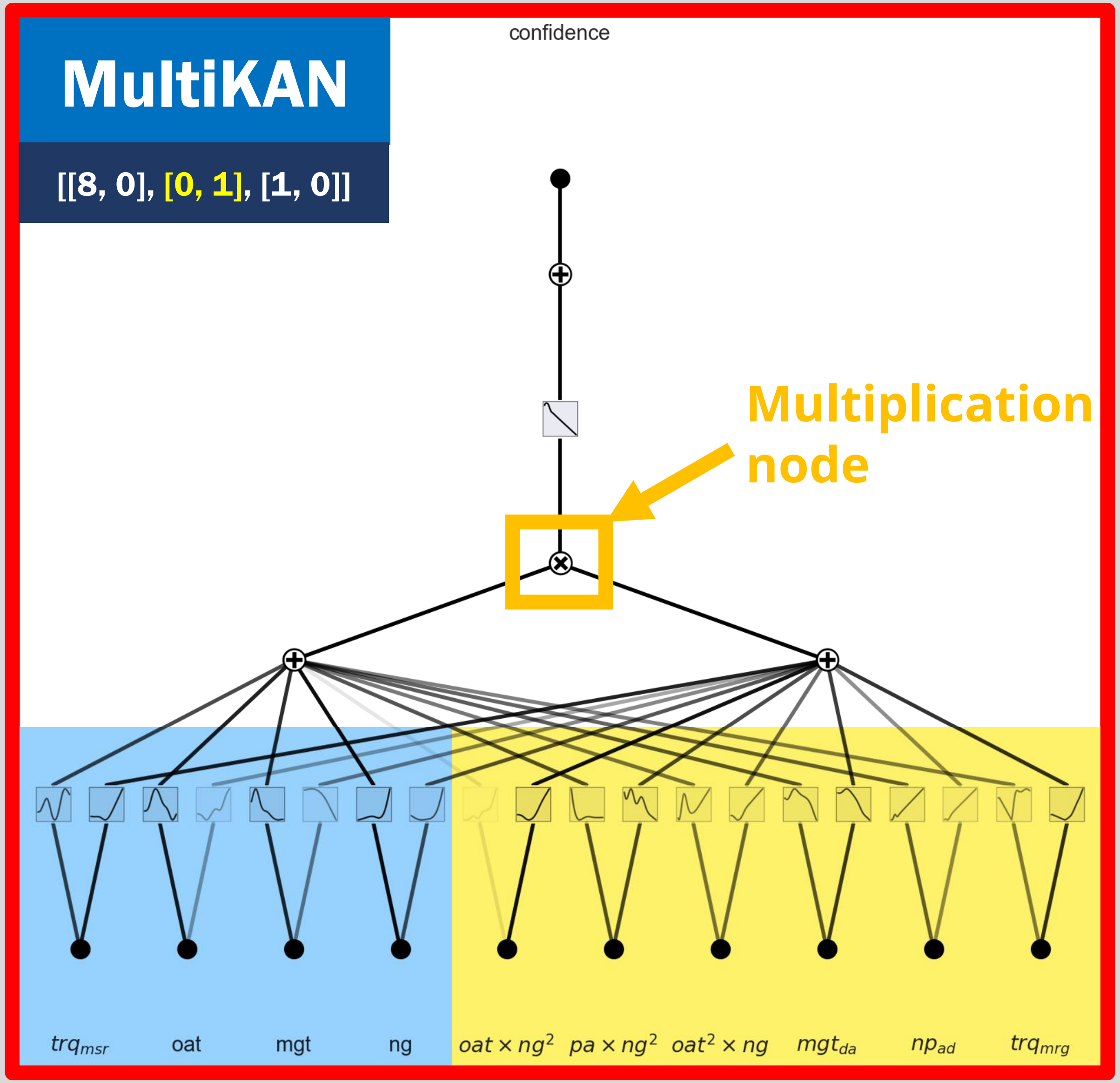


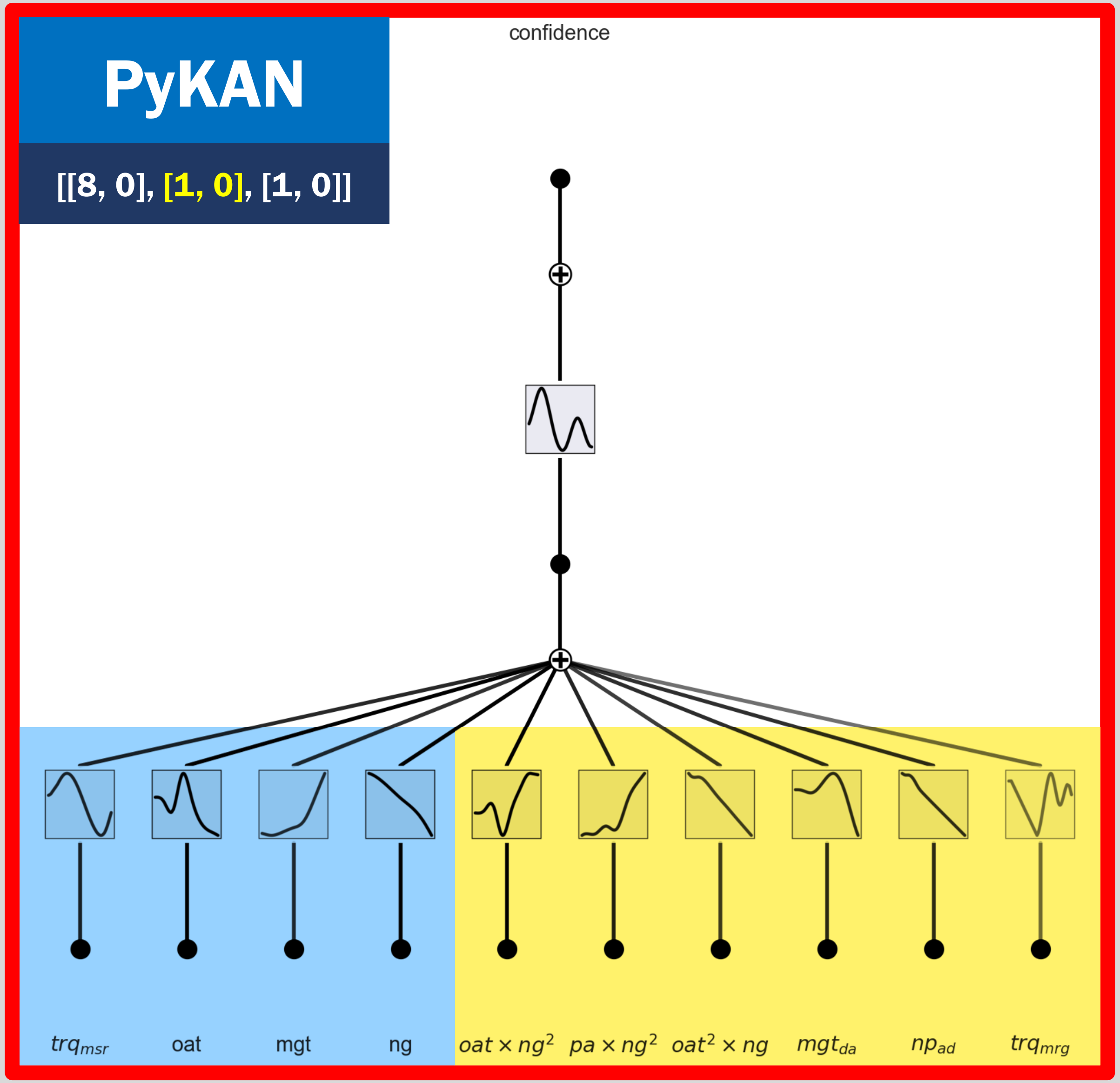
*regression*

# Some inference examples...

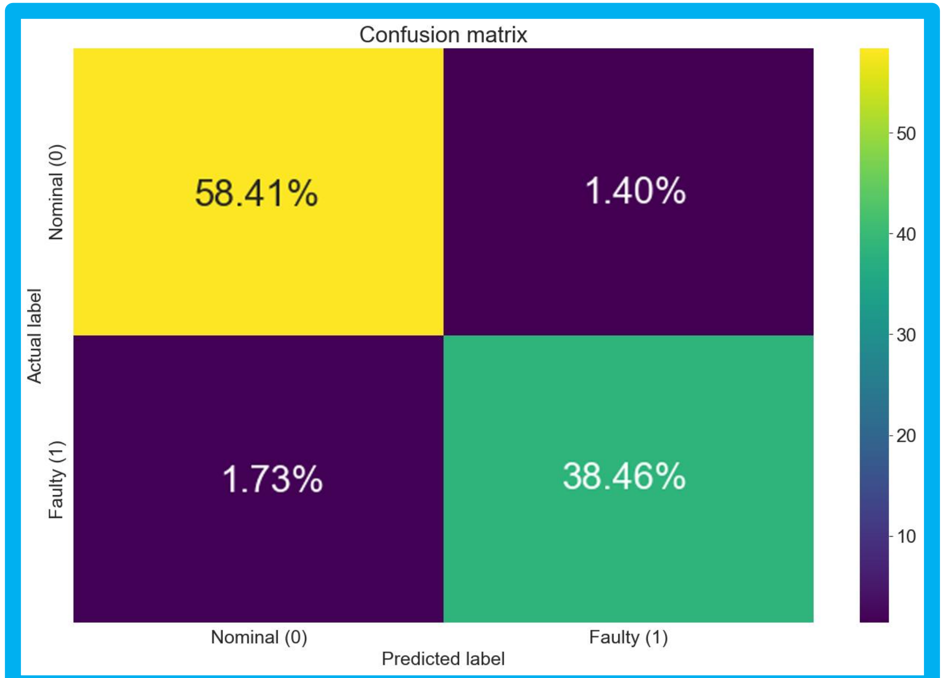


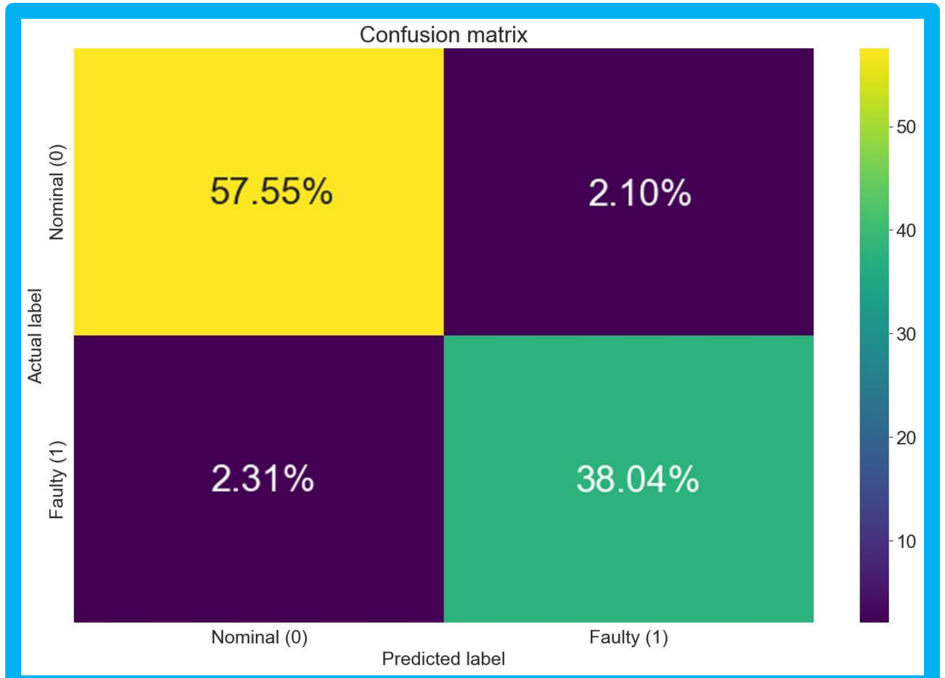
# Fault Detection with PyKAN



# Results of Fault Detection with PyKAN

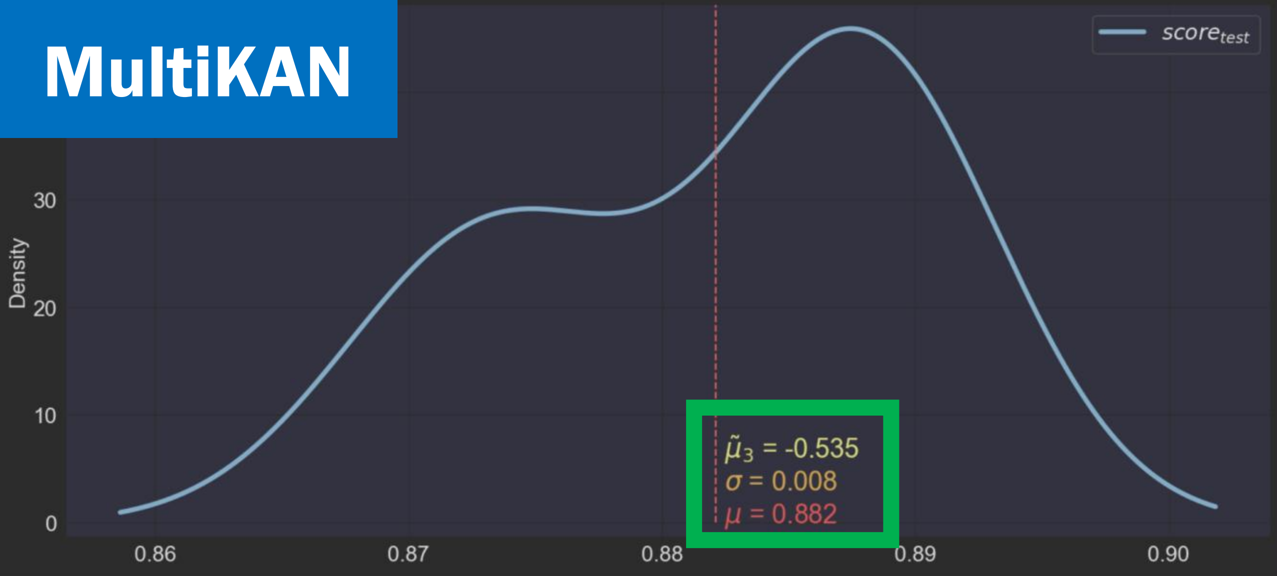


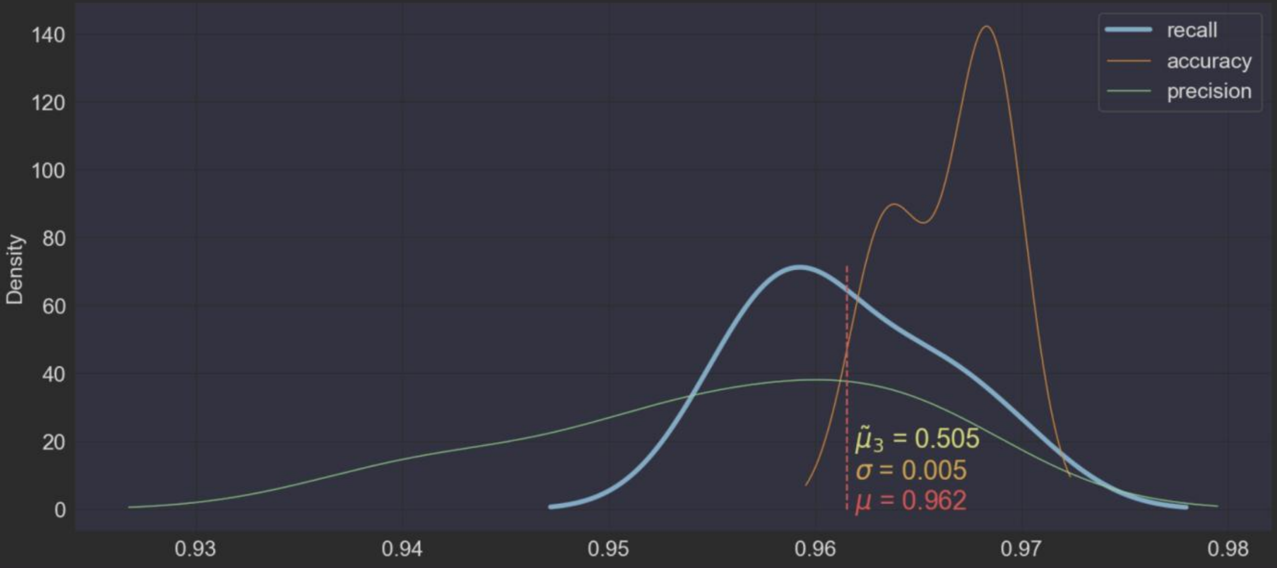




| Model | MultiKAN | | PyKAN | |
|---|---|---|---|---|
| **Avg Score** | 0.819 → | **0.891** | 0.815 → | **0.834** |
| **Accuracy** | 0.952 → | **0.969** | 0.950 → | **0.956** |
| **Precision** | 0.938 → | **0.960** | 0.933 → | **0.948** |
| **Recall** | 0.944 → | **0.957** | 0.945 → | **0.943** |
| **F1-score** | 0.941 → | **0.961** | 0.939 → | **0.945** |

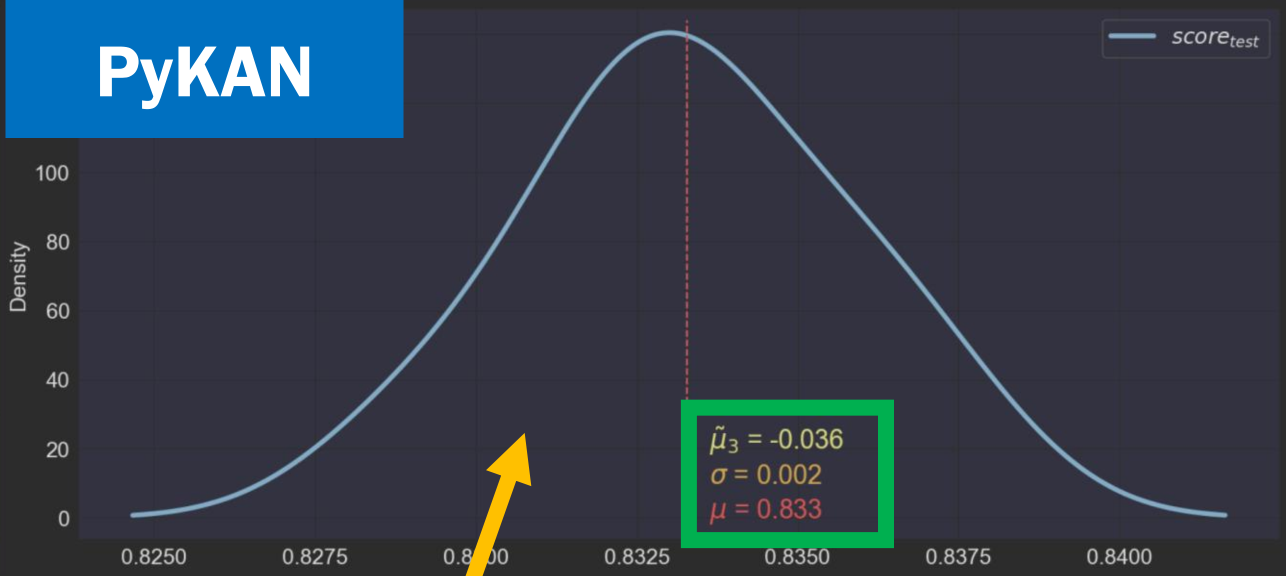
# Results of 10 trainings

## MultiKAN

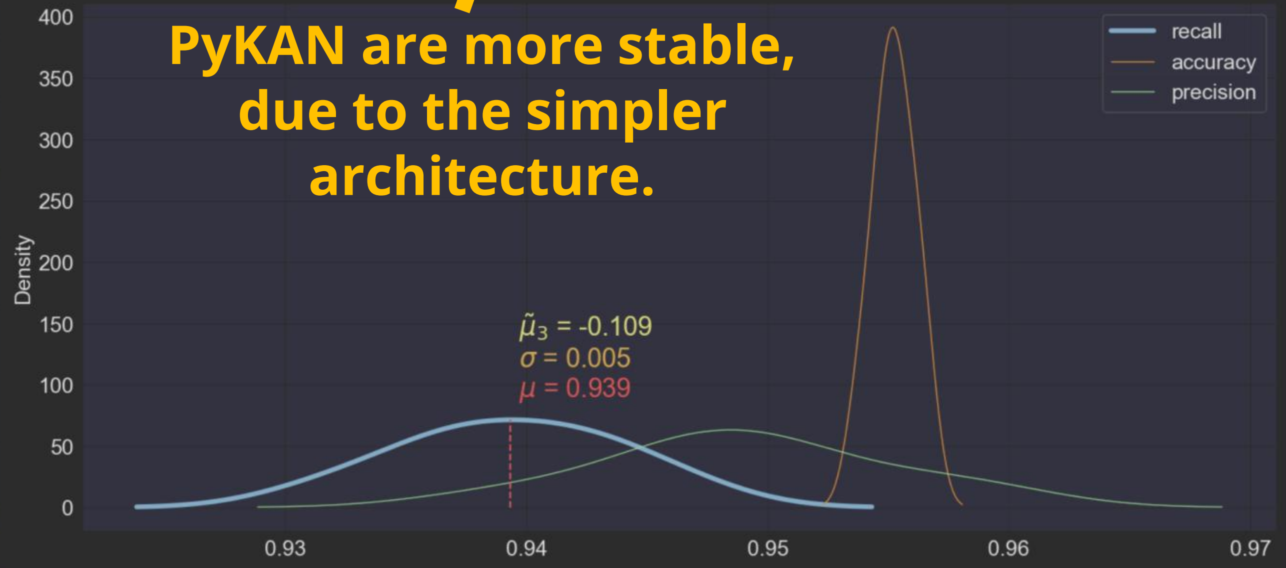



## PyKAN




**PyKAN are more stable, due to the simpler architecture.**



# PHM North America 2024 Challenge

**Feature selection**

Which subset of features we chose and how?

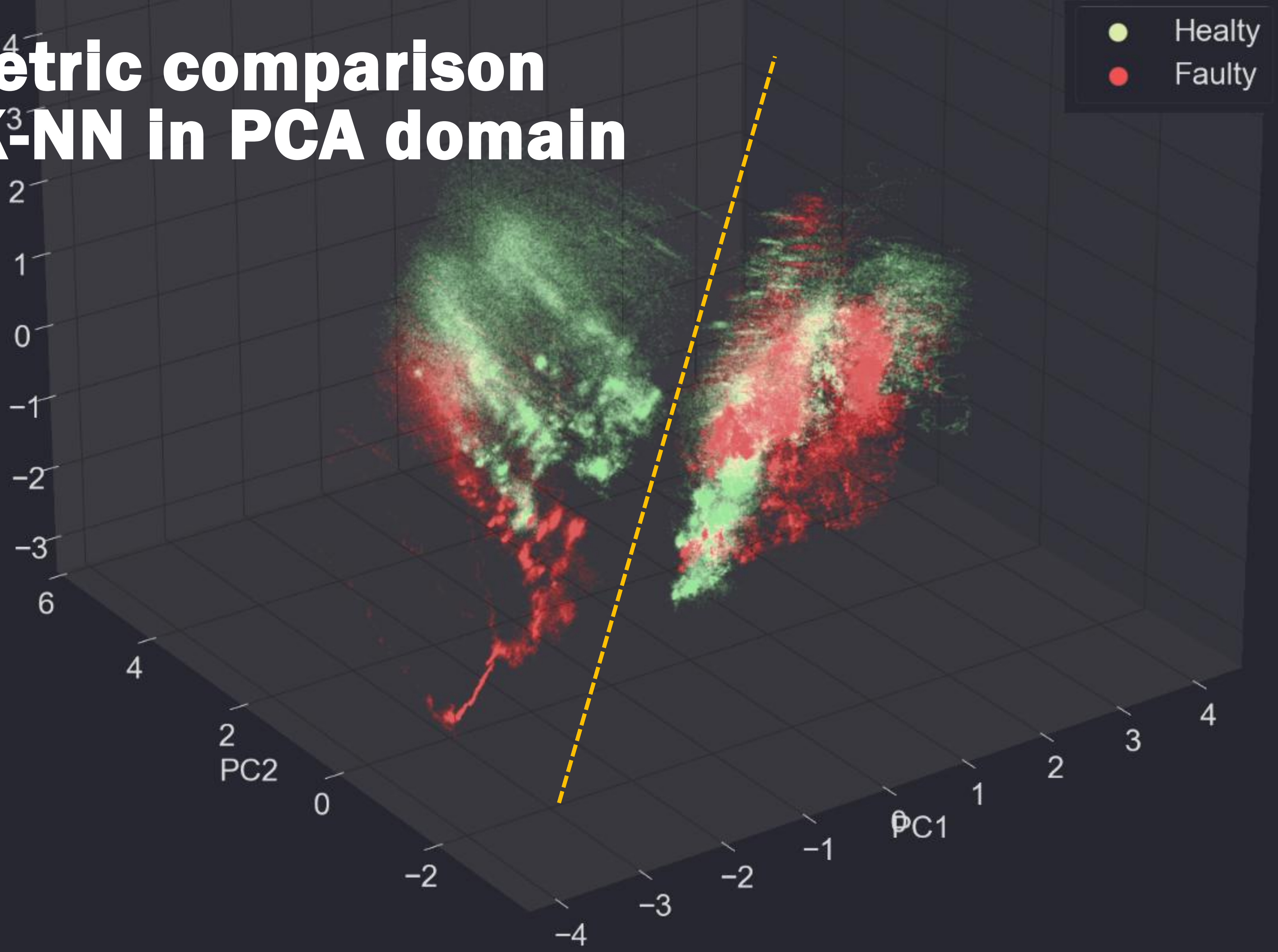
**Probabilistic regression and fault detection**

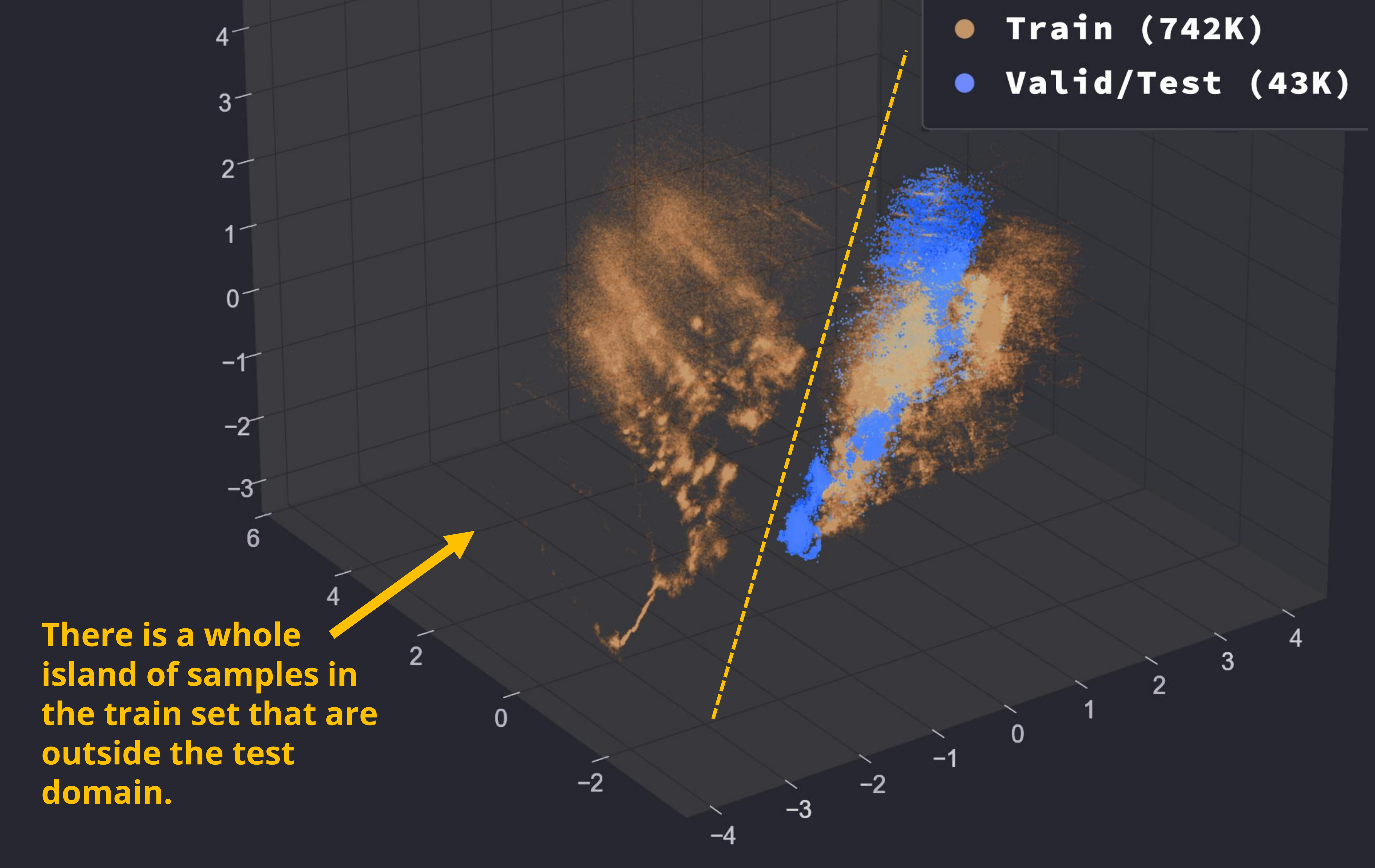
What are the results of our experiments?

**Qualitative Estimation**

Does the model generalize?

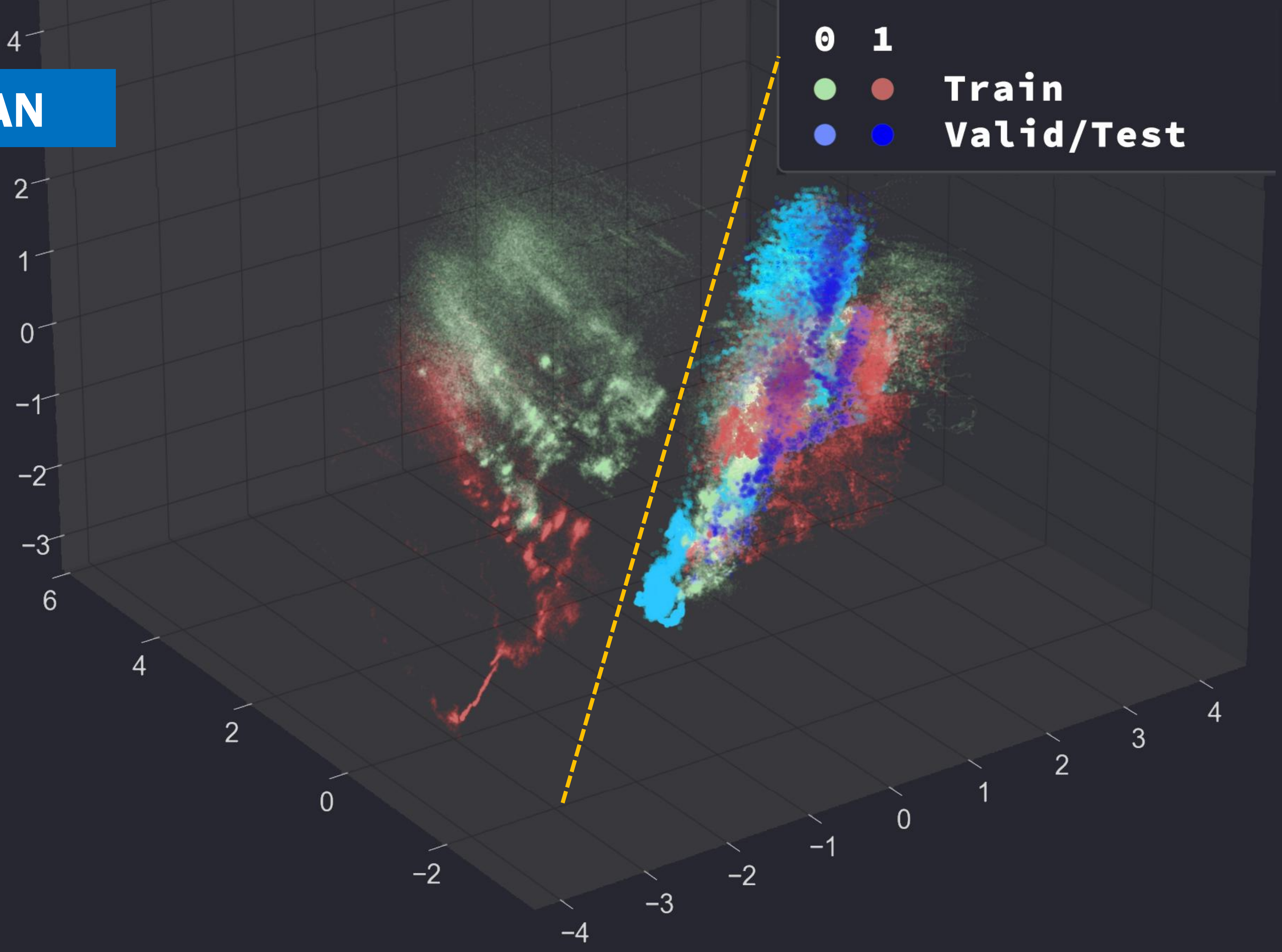
# Geometric comparison with K-NN in PCA domain



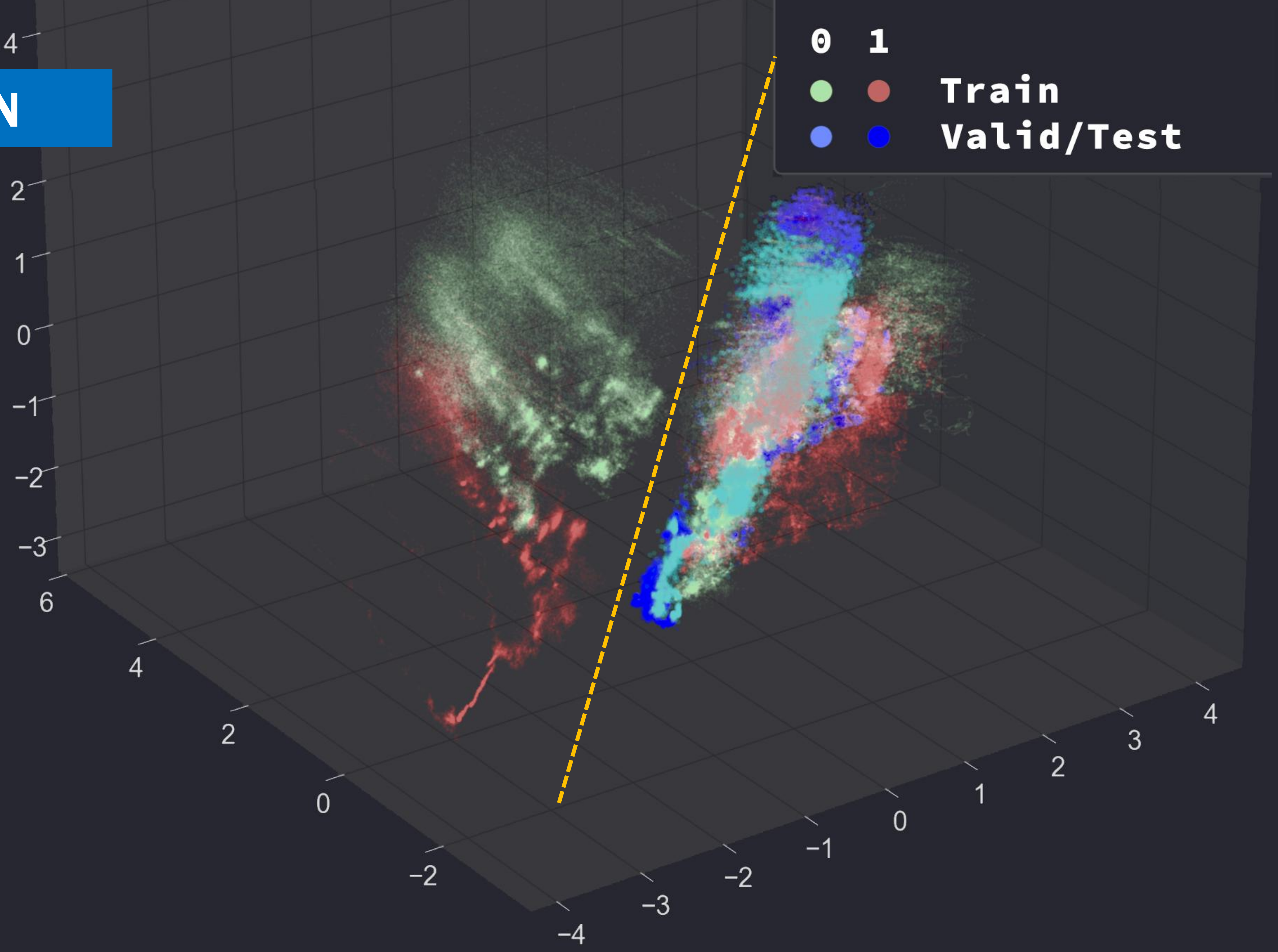


Train (742K)
Valid/Test (43K)
4
3
2
1
0
−1
−2
−3
6
4
2
0
−2
−4
−3
−2
−1
0
1
2
3
4
There is a whole island of samples in the train set that are outside the test domain.

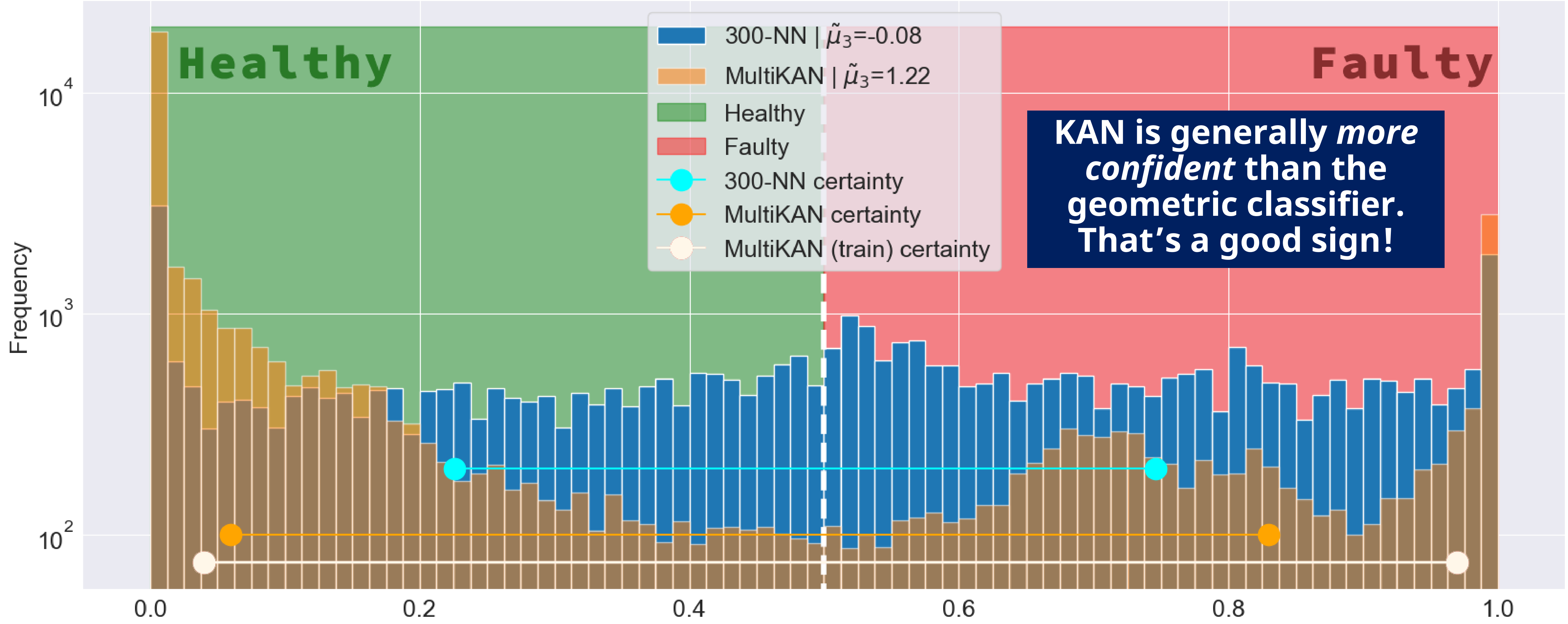
# MultiKAN



# 300-NN



# MultiKAN vs 300-NN: a confidence comparison

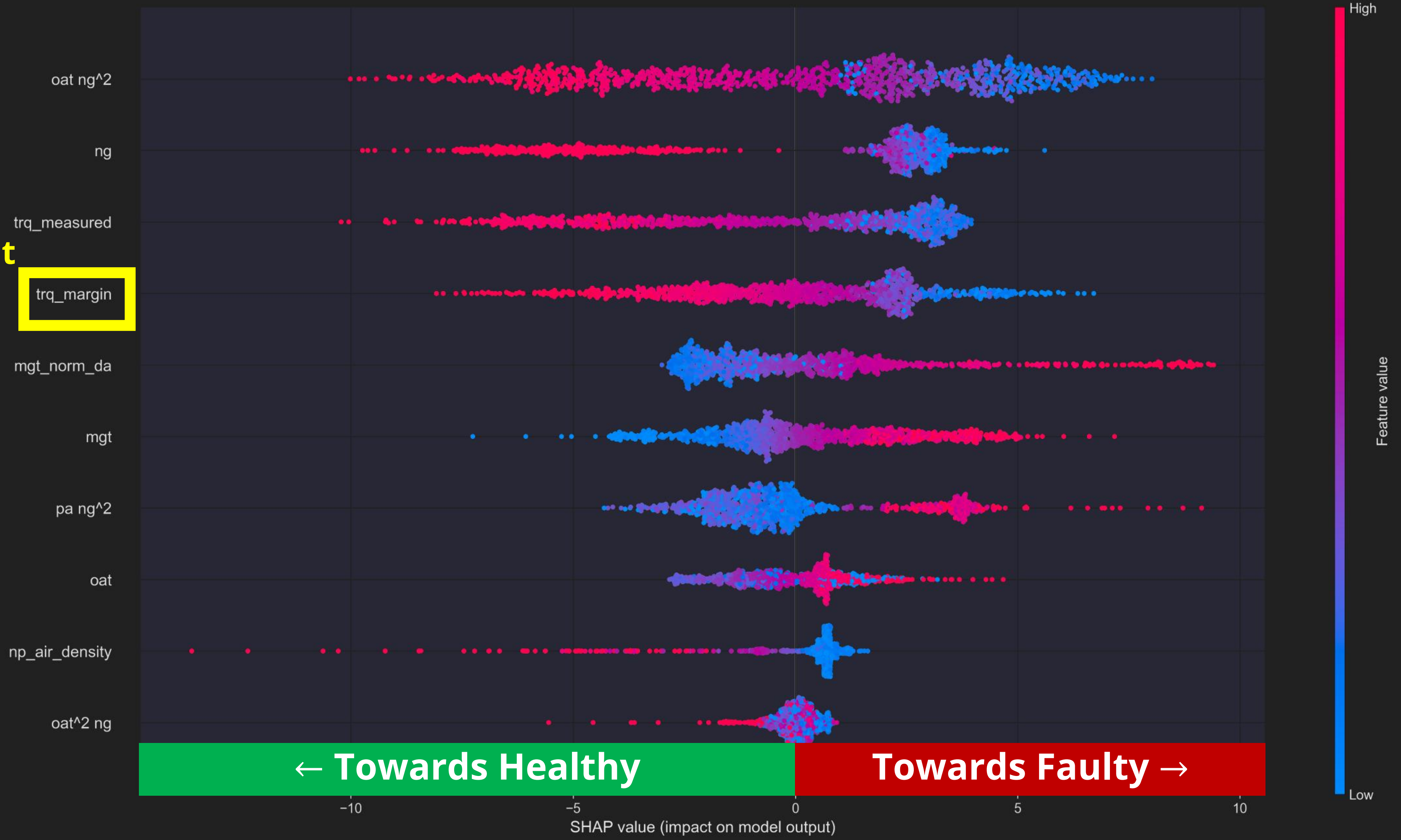



**The alignment score between *PyKAN* and *300-NN* is 0.58**

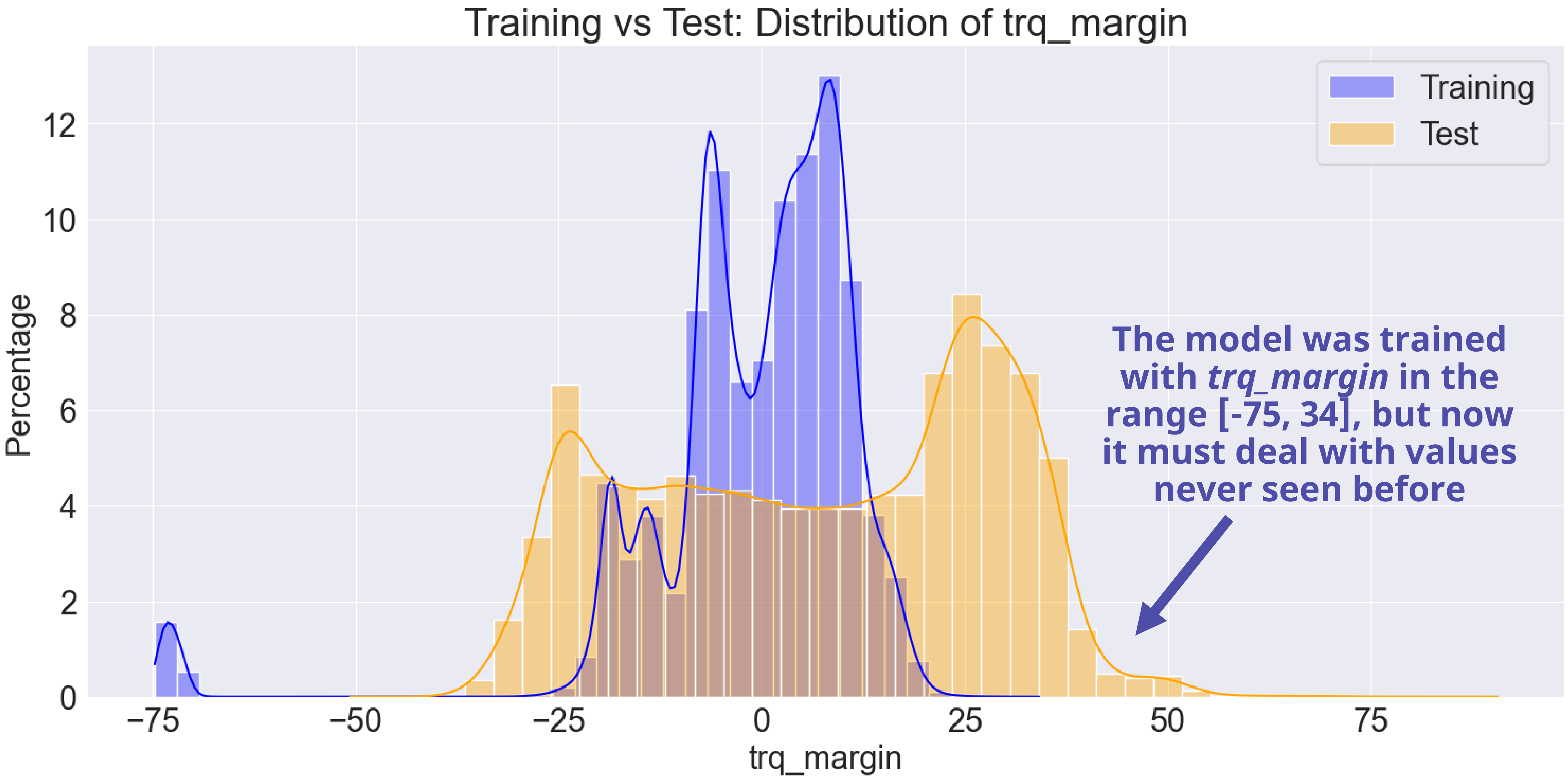
$$1 - \sum_{i=1}^{N} \frac{|y_i^{KNN} - y_i^{PyKAN}|}{N}$$

# What's driving the model's decision?

The output of the regressor

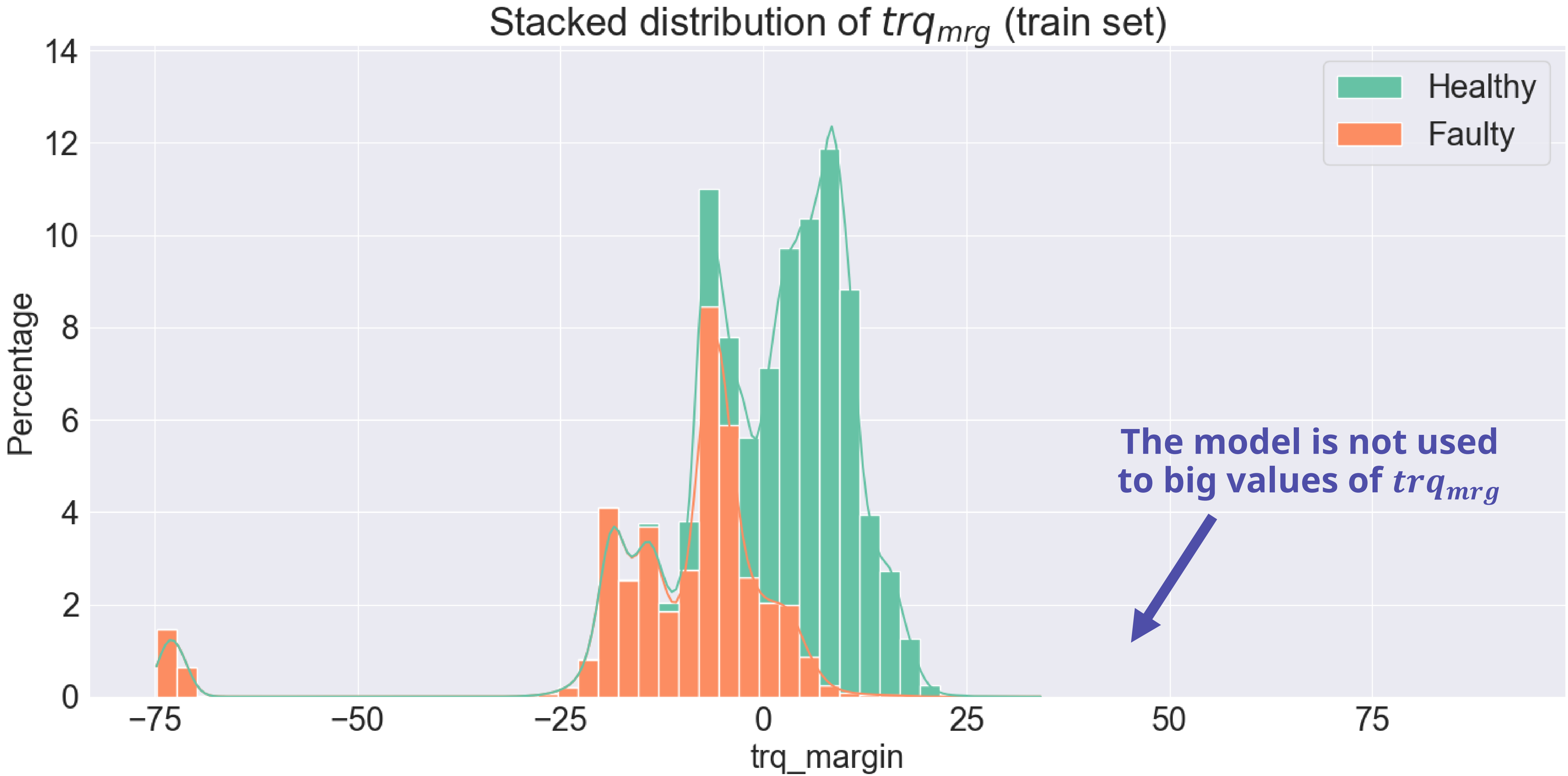


# Data drift analysis



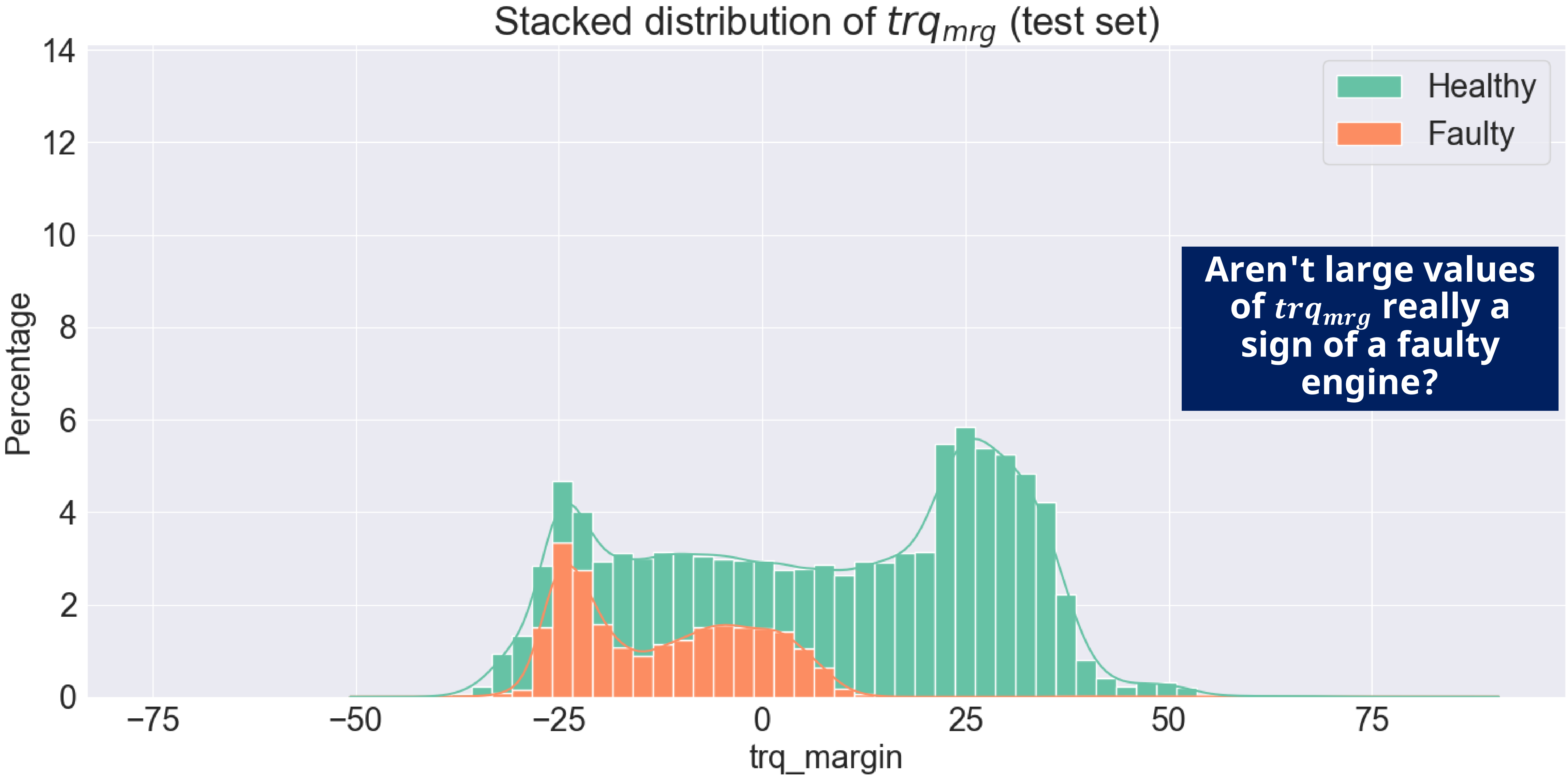

**The model was trained with *trq_margin* in the range [-75, 34], but now it must deal with values never seen before**

# Data drift analysis



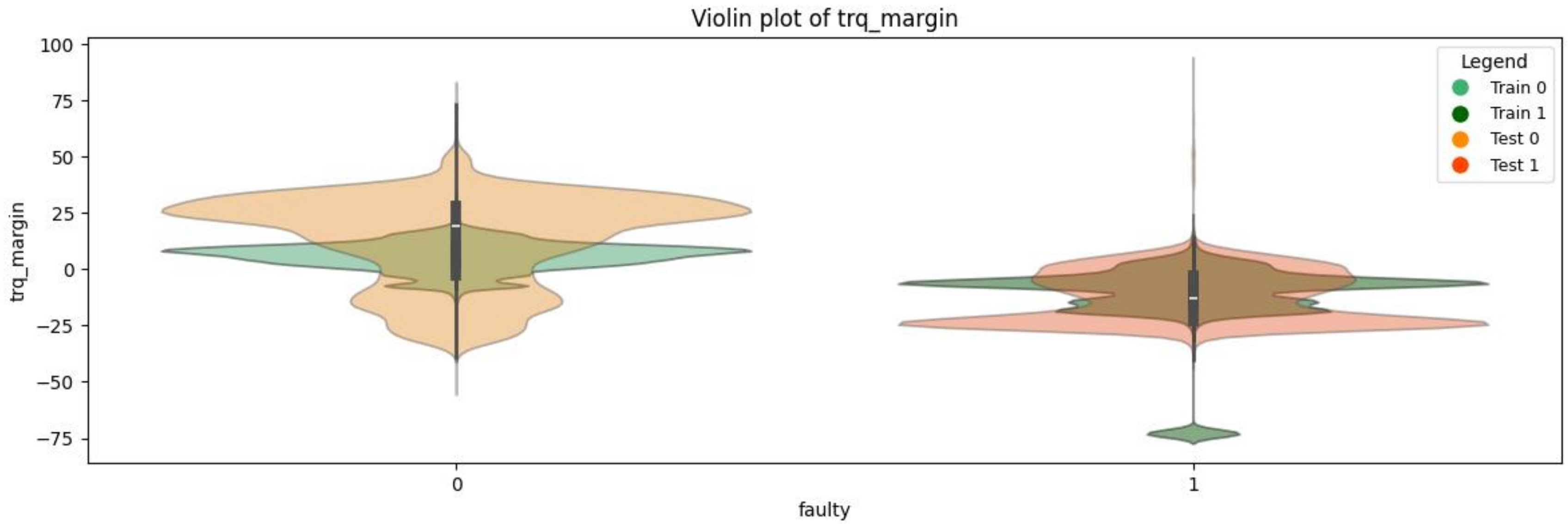

**The model is not used to big values of $trq_{mrg}$**

# Data drift analysis




**Aren't large values of $trq_{mrg}$ really a sign of a faulty engine?**

# Healthy vs Faulty in train and test





- To assign labels to the test data, we used the predictions of our classifier
- Violin plots compare the distributions of features between training and test

- We are tying to understand **if pattern observed in training are reproduced in test set**

# Outlier analysis in $trq_{mrg}$

- Outlier identification criterion: Interquartile range (IQR)

$$IQR = Q3 - Q1$$

$$\left.\begin{aligned} Lower\ bound &= Q1 - 1.5 \times IQR \\ \text{Upper bound} &= Q3 + 1.5 \times IQR \end{aligned}\right\}$$

**Values below the lower bound and above the upper bound are outliers**

This method makes no assumptions about the kind of the distribution

- Examples of outliers in training set

**In the training set, 15,746 outliers were identified in trq_margin, all with negative values, related to faulty engines**

| ID | trq_margin | trq_target | trq_measured | faulty |
|---|---|---|---|---|
| 30 | -71.71 | 78.15 | 22.11 | 1 |
| 38 | -72.82 | 80.10 | 21.77 | 1 |
| 65 | -71.48 | 71.85 | 20.49 | 1 |
| 76 | -72.34 | 78.45 | 21.70 | 1 |
| 210 | -71.73 | 78.09 | 22.08 | 1 |

# Outlier analysis in $trq_{mrg}$

- Example of outlier in test set

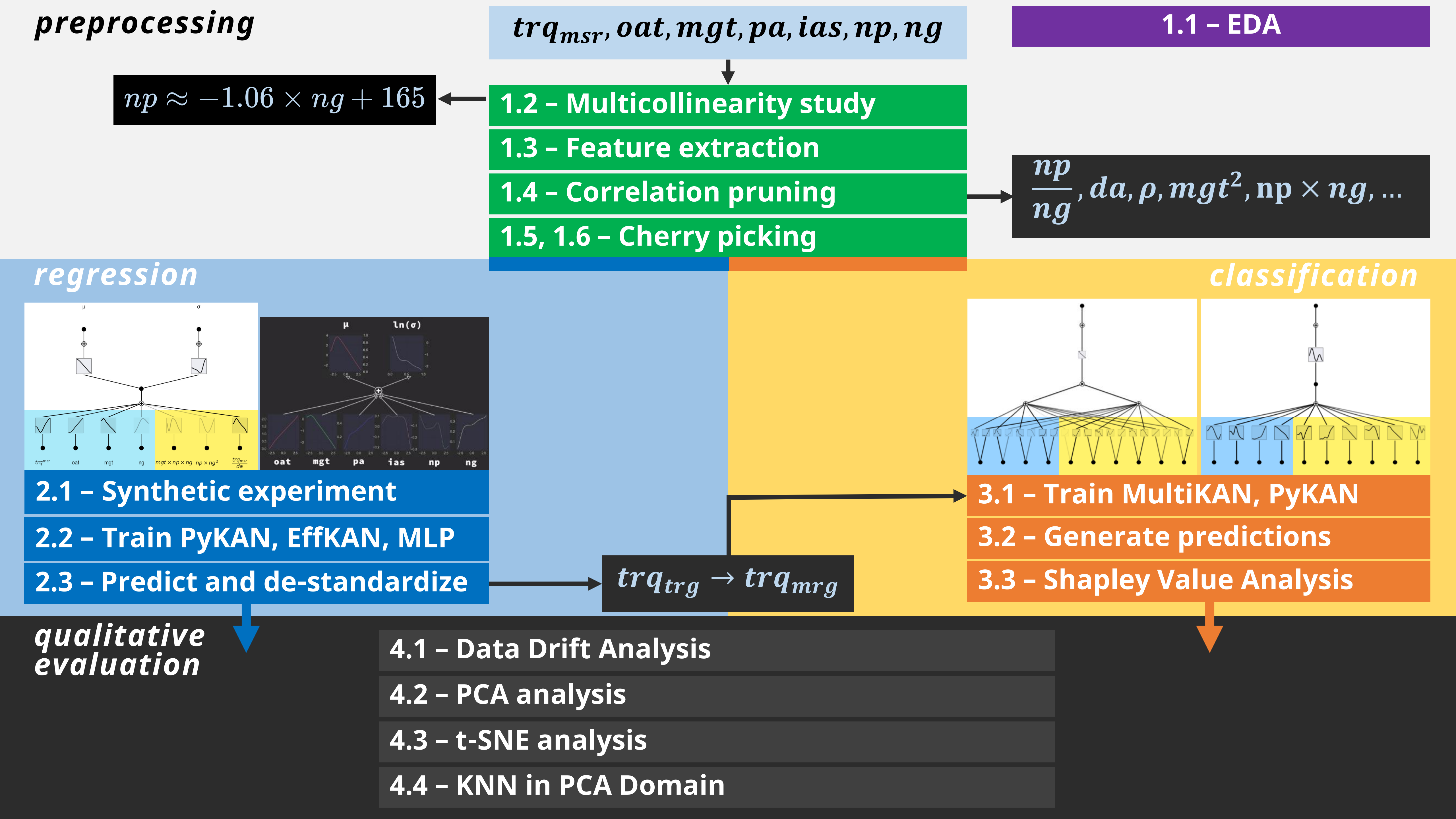
| ID | trq_margin | trq_target | trq_measured | faulty |
|---|---|---|---|---|
| 753 | 90.60 | 50.37 | 96.0 | 0.9999 |

One outlier, with an extremely high $trq_{mrg}$, was identified. The model classifies it, with high confidence as a failure.

- Comparison of other features

| Feature | Value | Range 5 - 95% |
|---|---|---|
| oat | 19.5 | 0.0 – 25.75 |
| mgt | 671.0 | 542.96 – 656.9 |
| pa | 187.76 | 144.78 – 855.41 |
| ias | 27.25 | 3.85 – 125.63 |
| np | 100.24 | 99.69 – 100.22 |
| ng | 98.65 | 90.18 – 97.58 |

In addition, some features are also above the 95% percentile, indicating that this engine is under ubnormal condition



preprocessing
$trq_{msr}, oat, mgt, pa, ias, np, ng$
1.1 – EDA
$np \approx -1.06 \times ng + 165$
1.2 – Multicollinearity study
1.3 – Feature extraction
1.4 – Correlation pruning
1.5, 1.6 – Cherry picking
$\frac{np}{ng}, da, \rho, mgt^2, \mathrm{np} \times ng, \ldots$
regression
classification
μ
ln(σ)
oat
mgt
pa
ias
np
ng
2.1 – Synthetic experiment
2.2 – Train PyKAN, EffKAN, MLP
2.3 – Predict and de-standardize
$trq_{trg} \rightarrow trq_{mrg}$
3.1 – Train MultiKAN, PyKAN
3.2 – Generate predictions
3.3 – Shapley Value Analysis
qualitative evaluation
4.1 – Data Drift Analysis
4.2 – PCA analysis
4.3 – t-SNE analysis
4.4 – KNN in PCA Domain